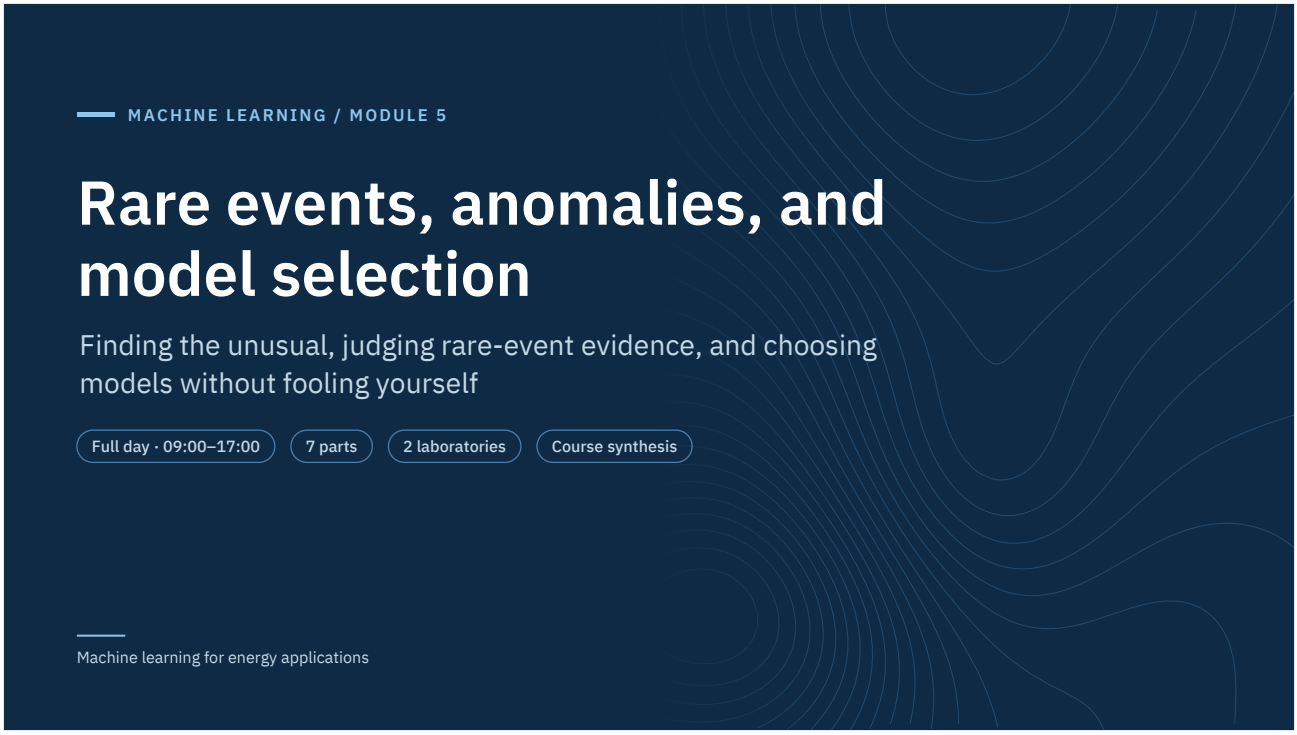
Rare events, anomalies, and model selection



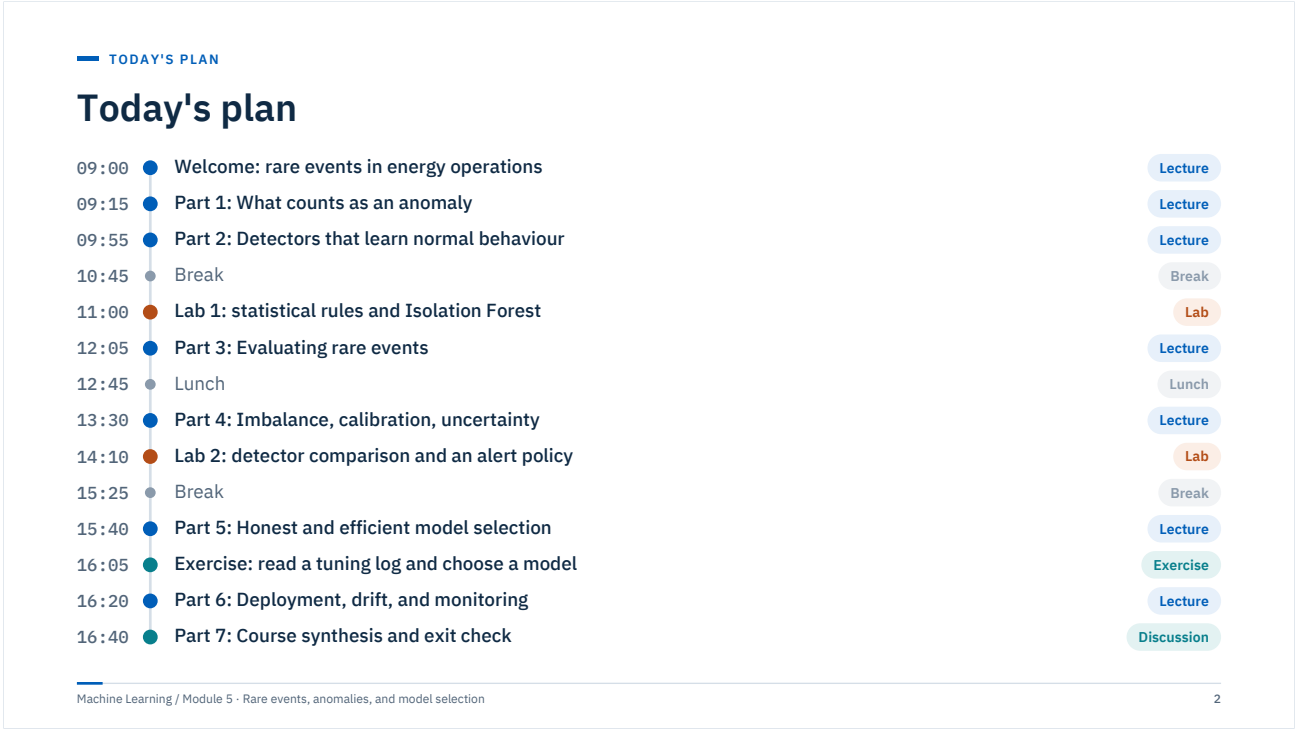
Welcome to the final day of the machine learning course. Today joins three threads that meet in almost every monitoring project in energy operations: finding unusual behaviour when labels are scarce, judging a detector when events are rare, and choosing models without fooling ourselves during tuning.

Prerequisites: classification metrics, pipelines and cross-validation (Module 1), ensembles (Module 2), clustering (Module 3) and PCA (Module 4). We refer back to each of them rather than re-teaching them.

Set the tone early. We treat anomaly detection as a ranking and investigation problem. A high anomaly score does not itself diagnose an equipment fault; it tells an engineer where to look first. Every number shown today comes from synthetic teaching data or from the course well-log file used in the labs, and none of it is a claim about field performance.

The day ends with a synthesis of the whole course and a final review exercise, so keep a note of questions from earlier modules. [2 min]

Today's plan



Walk through the shape of the day. The morning builds the detectors: what an anomaly is, robust statistical baselines, and four families of learned detectors. Lab 1 applies the statistical rules and Isolation Forest to the course well-log file (notebook Parts 1 to 3).

Before lunch we switch from finding unusual rows to judging a detector when events are rare. After lunch we cover class imbalance, probability calibration, and honest uncertainty with few events. Lab 2 then completes the notebook (Parts 4 to 6: LOF, One-Class SVM and a method comparison) and ends with a written alert policy.

The last teaching block covers model selection (tuning, nested cross-validation, statistical comparison of models) and ends with a paper exercise on reading a tuning log. Then deployment and drift monitoring, and we finish with a synthesis of Modules 1 to 5, a final review exercise and the exit check.

Timing: about 4 hours 25 minutes of teaching including in-class exercises, 2 hours 20 minutes of labs, two 15-minute breaks and a 45-minute lunch. If you run late, shorten Part 6 by moving the monitoring-case table to the notes. [2 min]

What you will be able to do by 17:00

LEARNING OUTCOMES

What you will be able to do by 17:00

By the end of the day you will be able to:

1. Match an anomaly problem to its setting and to its type: point, contextual, or collective.
2. Build robust statistical baselines and compare Isolation Forest, LOF, One-Class SVM, and reconstruction error.
3. Evaluate rare-event detectors with precision-recall, event-level metrics, calibration, and honest intervals.
4. Set and document an alert threshold from costs or from review capacity.
5. Run a bounded, leakage-free tuning experiment and compare models with nested CV and paired folds.
6. Plan drift monitoring and connect the full workflow of Modules 1 to 5.

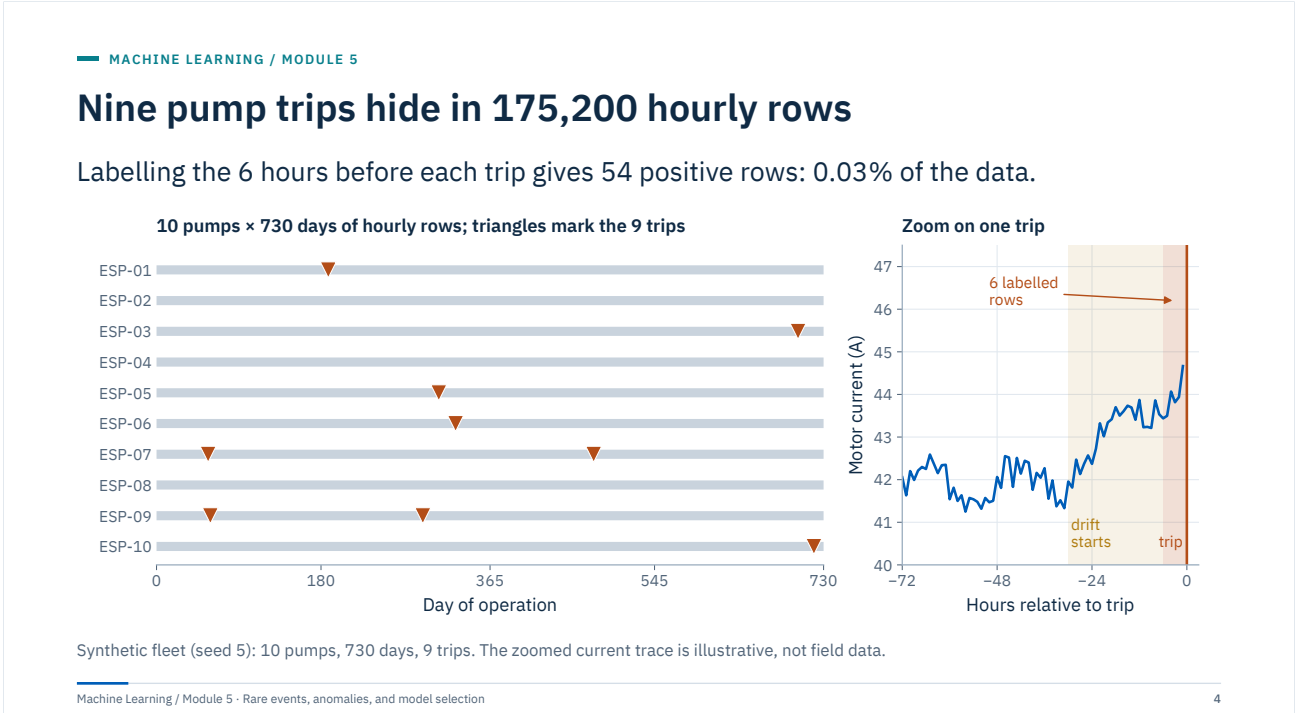
Machine Learning / Module 5 · Rare events, anomalies, and model selection 3

Read the outcomes aloud and point out that each one is observable: at the end of the day learners should be able to do something, not just recognize a term.

Outcome 1 is about vocabulary that changes decisions: whether you have labels decides the method, and the type of anomaly decides the features. Outcome 2 covers the detectors used in both labs. Outcome 3 is the heart of the day, because most rare-event projects fail at evaluation rather than at modelling. Outcome 4 is assessed directly by the alert policy written in Lab 2. Outcome 5 applies to every supervised model in the course, not only to rare events. Outcome 6 closes the course.

Ask the room: who has seen an alarm system that operators learned to ignore? Keep one or two examples in mind; we return to alert fatigue in Part 3. [2 min]

Nine pump trips hide in 175,200 hourly rows



Introduce the motivating case: a fleet of ten electric submersible pumps logged hourly for two years. That is 10 × 730 × 24 = 175,200 rows, but only nine trips. If an engineer labels the six hours before each trip as positive, there are 54 positive rows, about 0.03% of the data.

The right panel zooms on one trip. The motor current drifts upward about 30 hours before the trip, but only the last six rows carry the label. This mismatch between physical precursors and labels is typical and it shapes the whole day. Stress that the trace is synthetic; we make no claim about how real pumps fail.

Four questions follow from this picture. Without reliable labels, how do we find unusual behaviour? With nine events, how do we evaluate a detector honestly? How do we tune without fitting noise in so few events? And after deployment, how do we know the model still works?

Ask: which of the nine trips would you use for validation, and which for testing? Note that two pumps had two trips each, so trips are not fully independent. [3 min]

Warm-up: judge a detector that never alerts

PRACTICE

3 min Pairs

Warm-up: judge a detector that never alerts

Same fleet: 175,200 hourly rows, of which 54 are labelled positive (6 per trip).

1. What accuracy does a model that never alerts achieve?
2. How many of the nine trips does it catch?
3. Which two numbers would you ask for instead?

Give pairs two minutes, then take answers.

Answer 1: the never-alert model is correct on every negative row, so accuracy = 1 − 54 / 175,200 = 1 − 0.000308 = 0.99969, which rounds to 99.97%.

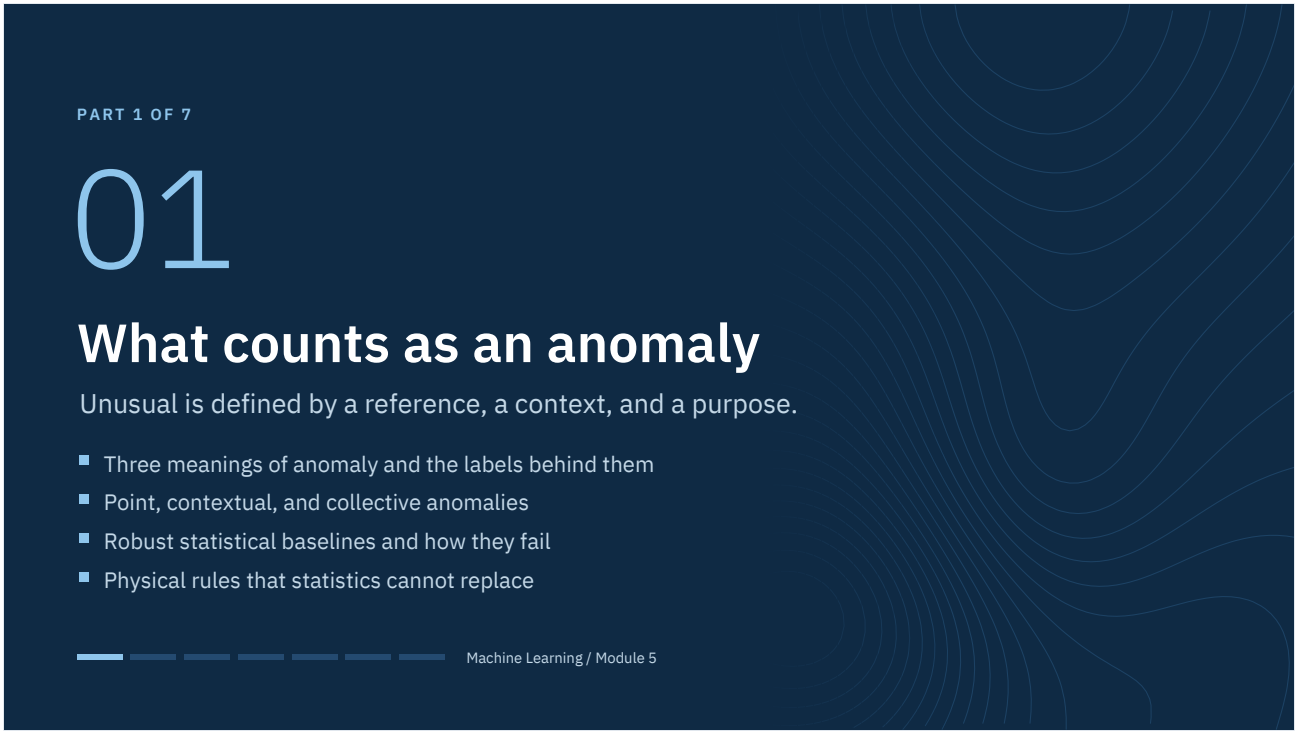
Answer 2: it catches none of the nine trips. Event recall is 0 of 9.

Answer 3: ask for event recall (the share of trips warned in time, with the warning window stated) and for false alerts per exposure unit, for example per pump-month. Good answers also mention lead time and the review capacity of the team. Precision at the chosen threshold is acceptable if the answer explains that it depends on prevalence.

Common misconception: "99.97% accurate, so it must be excellent." With this prevalence, accuracy measures how rare the events are, not how good the detector is. A second misconception is that row-level recall answers the operator's question; we will see in Part 3 that row counts and event counts can tell different stories.

Transition: before evaluating detectors we need to be precise about what "anomaly" means. [3 min]

What counts as an anomaly



This part takes about 40 minutes. It establishes vocabulary and the simplest defensible baselines. Every learned detector in Part 2 should be compared against these baselines, just as every supervised model in Module 1 was compared against a dummy or linear baseline.

Emphasize that the word anomaly hides three different questions. Unusual compared with what reference data? In what context (operating regime, time of day, depth interval)? And for what purpose (data quality review, early warning, investigation ranking)? The answers decide the method and the evaluation.

The part ends with a short hand calculation that shows why robust statistics matter even for tiny samples. [1 min]

Anomaly has several meanings

WHAT COUNTS AS AN ANOMALY

Anomaly has several meanings

Setting	Training information	Typical question
Outlier detection	Mixed, mostly ordinary observations	Which training observations differ?
Novelty detection	A reference population intended to be normal	Does a new observation depart from normal?
Supervised rare-event prediction	Examples labelled by event outcome	How likely is the labelled event?

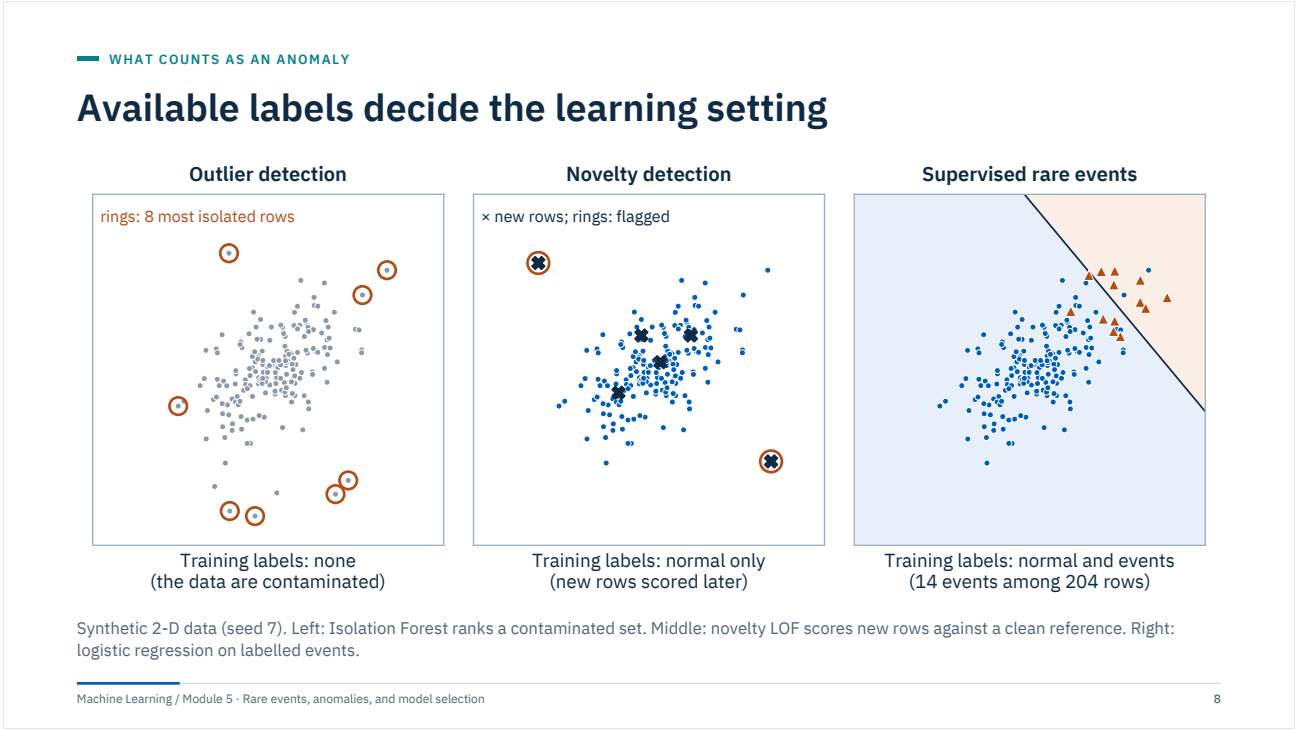
Machine Learning / Module 5 · Rare events, anomalies, and model selection 7

Walk through the three rows. In outlier detection the training data themselves contain the unusual rows, and the task is to rank them. In novelty detection a reference period is believed to be normal (for example a commissioning period reviewed by engineers), and new rows are compared with it. In supervised rare-event prediction we have labels for events, even if few, and we estimate their probability.

Context matters in all three. A pressure spike may be normal during start-up but unusual during steady operation. Collective anomalies occur over a sequence even if individual values look plausible. Include the operating regime in every review.

Ask: which setting fits the lab well-log file? Answer: outlier detection, because we score the same rows we fit on and have no reviewed labels. Which fits the pump fleet? Potentially all three, depending on what has been labelled and reviewed. [3 min]

Available labels decide the learning setting



Use the three panels to make the table concrete. Left: all rows are grey because nobody knows which are unusual. The detector ranks the rows it was trained on and the orange rings mark the eight most isolated. There is no ground truth, so the ranking can only be reviewed, not scored.

Middle: the blue reference set was curated to be ordinary. New rows (crosses) are scored against it; two fall outside and are flagged. Novelty detection is only as good as the curation of the reference period.

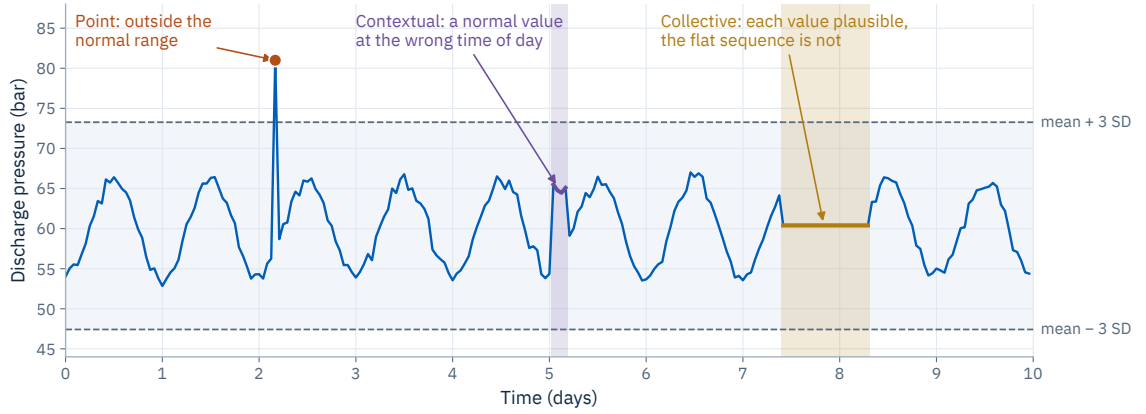
Right: fourteen labelled events allow a supervised classifier. Now we can estimate precision and recall, but with so few events the estimates are uncertain (Part 4) and the classifier learns only the event types seen before.

A common error is to call any of these "unsupervised learning" without saying which. The choice also decides the scikit-learn API: LocalOutlierFactor behaves differently in outlier and novelty mode, as the lab will show. [3 min]

Anomalies come as points, contexts, or collectives

WHAT COUNTS AS AN ANOMALY

Anomalies come as points, contexts, or collectives



Synthetic hourly discharge pressure (seed 11) with an injected spike, four peak-level hours at trough time, and a 22-hour flat segment.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 9

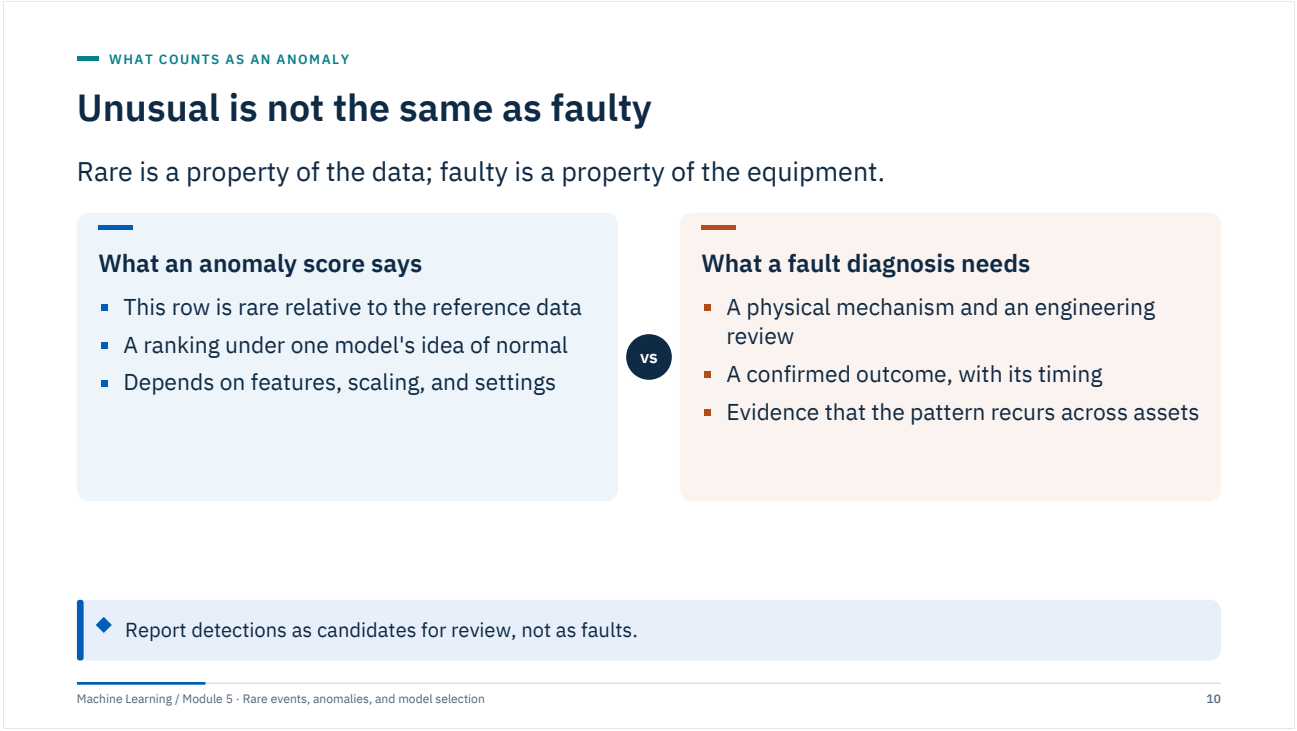
Point anomaly: a single value outside the normal range, like the spike on day 2. Every method in this module can find it, including the simple mean ± 3 SD band shown in grey.

Contextual anomaly: the values on day 5 are ordinary pressures, but they occur at the time of day when pressure is normally at its trough. Only a method that knows the context (time of day, operating mode, pump speed) can flag it.

Collective anomaly: from day 7.4 the sensor repeats one plausible value for 22 hours. No single value is unusual; the sequence is. Stuck sensors, frozen communication links and flat-lined historians produce exactly this pattern.

Note that the contextual and collective anomalies sit well inside the global band. Ask: which features would expose each type? Expected answers: hour of day or a seasonal forecast residual for the contextual case, and rolling variance or a count of repeated values for the collective case. We return to both in Part 2. [3 min]

Unusual is not the same as faulty



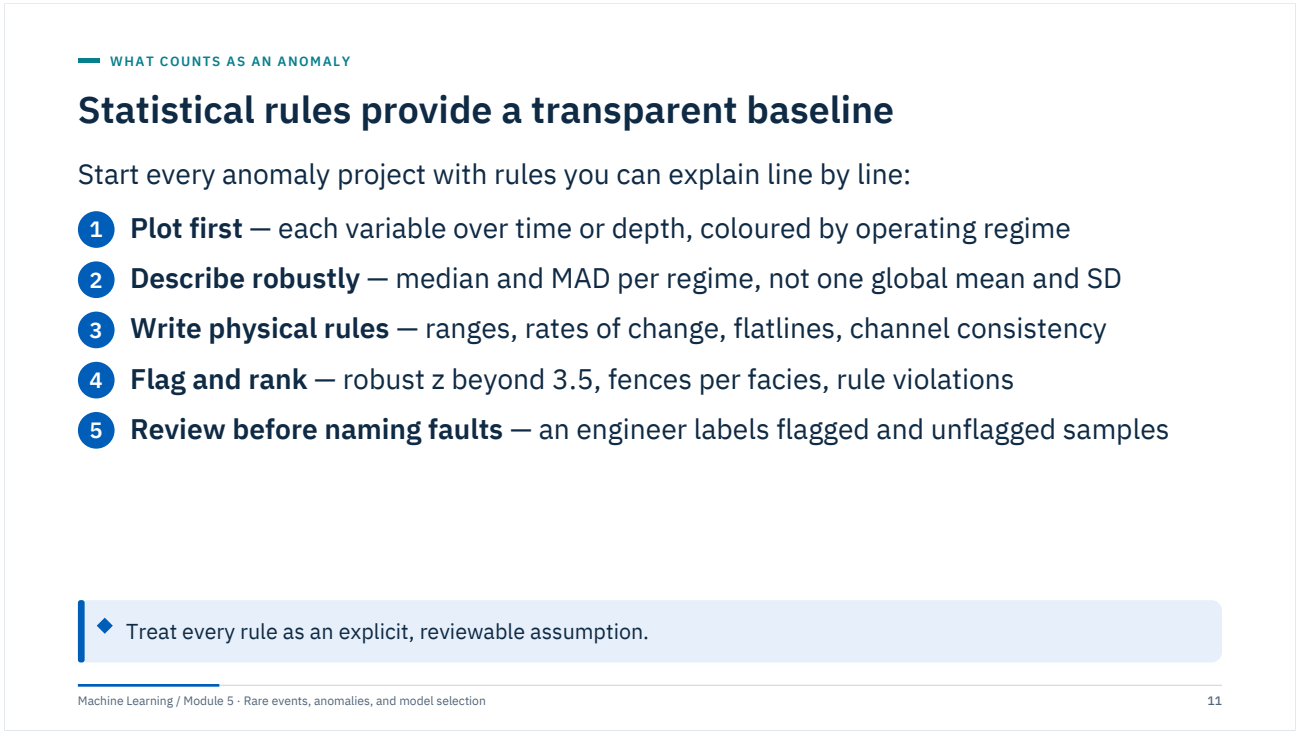
This distinction protects both the analyst and the operator. An anomaly score is a statement about the data and the model: this row is rare under the chosen features and reference. It is not a statement about equipment health.

Many unusual rows are ordinary operations that happen rarely: a planned shutdown, a well test, a new operating regime, a recalibrated sensor. Many genuine faults look ordinary in the chosen features. A diagnosis needs a physical mechanism, confirmation and timing, and it gains weight when the pattern recurs on other assets.

Practical wording for reports: "the detector ranked these intervals highest for review" rather than "the model found these faults". Keep this in mind for the lab write-up.

Ask: in your own data, what rare but normal operations would a detector flag? Typical answers: start-ups, well tests, maintenance, calibrations, logging-tool changes. [2 min]

Statistical rules provide a transparent baseline



The baseline for anomaly detection is a set of transparent rules. First plot the data over time and by regime; many anomalies are obvious to the eye and many "anomalies" turn out to be regime changes. Then describe each variable with robust statistics, add physical range and rate-of-change rules, and review what is flagged.

A z-score threshold assumes a meaningful centre and spread; heavy tails and mixtures of regimes produce many false alarms, and extreme values inflate the spread itself (next slide). The median absolute deviation (MAD) is more robust but also has limits: it can be zero when more than half the values are identical, for example for a stuck or quantized sensor.

Keep physical unit checks separate from learned anomaly scores so that each can be reviewed and changed on its own. [2 min]

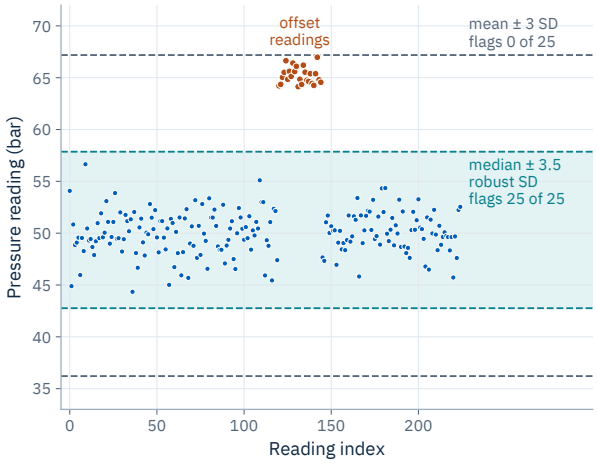
Outliers inflate the statistics meant to catch them

WHAT COUNTS AS AN ANOMALY

Outliers inflate the statistics meant to catch them

$$z = \frac{x - \bar{x}}{s} \qquad z_{\text{robust}} = \frac{x - \text{median}}{1.4826\,\text{MAD}}$$

- 25 offset readings raise the SD from about 2 to 5.2 bar
- Classical |z| > 3 flags none of them: masking
- Median and MAD barely move; robust |z| > 3.5 flags all 25



Synthetic readings (seed 3): 200 ordinary N(50, 2) and 25 offset N(65, 0.8).

Machine Learning / Module 5 · Rare events, anomalies, and model selection 12

Define the symbols: $\bar{x}$ is the mean, s the standard deviation, and MAD the median of the absolute deviations from the median. Multiplying the MAD by 1.4826 makes it estimate the standard deviation when the data are Gaussian. A robust cut-off of 3.5 is the value suggested by Iglewicz and Hoaglin (1993) for their modified z-score.

The numbers: the 225 readings have mean 51.70 and SD 5.16, so the classical upper limit is 67.2 bar and the largest offset reading has z = 2.96. None is flagged. The median is 50.31 and the MAD 1.45, so the robust limit is 57.9 bar and the smallest robust z among the offset readings is 6.4: all 25 are flagged, and no ordinary reading is.

This effect is called masking: the outliers inflate the statistics used to detect them. It happens whenever a sizable minority of rows is affected, for example a calibration offset lasting several days.

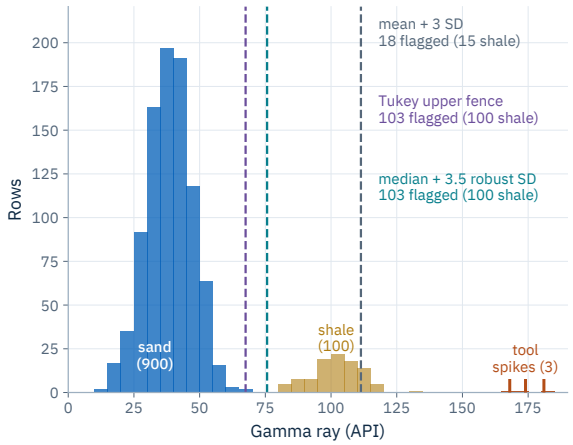
Ask: what would happen with a 50% offset? Answer: the median itself would move, so no single-population rule works; you need to split by period or regime. [3 min]

Univariate fences assume a single population

WHAT COUNTS AS AN ANOMALY

Univariate fences assume a single population

- Tukey fences: below Q1 − 1.5 IQR or above Q3 + 1.5 IQR
- A 10% shale population sits beyond both robust fences
- 100 of the 103 robust flags are valid shale, not tool error
- Set fences per facies or regime, or use multivariate methods



Synthetic GR-like mixture (seed 21): 900 sand rows N(38, 9), 100 shale rows N(102, 9), three tool spikes.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 13

Define IQR: the interquartile range, Q3 minus Q1. Tukey's box-plot fences sit 1.5 IQR beyond the quartiles.

Here the gamma-ray values come from two populations: 900 sand rows and 100 shale rows, plus three tool spikes. Both robust rules put their upper fence inside the gap between the populations (Tukey at 67.5 API, median + 3.5 robust SD at 75.7 API) and flag 103 rows each: the 3 spikes and all 100 shale rows. The classical mean + 3 SD rule sits at 111.4 API and flags 18 rows, 15 of them shale.

So robust statistics solve masking but not mixtures. A univariate fence assumes one population; geology rarely provides one. Remedies: compute fences within a facies or zone (Module 3 clustering can supply the regimes), transform skewed variables such as resistivity with a logarithm first, or use a multivariate detector.

You will see this effect on real data in Lab 1: most IQR flags in the course file come from high gamma ray. [3 min]

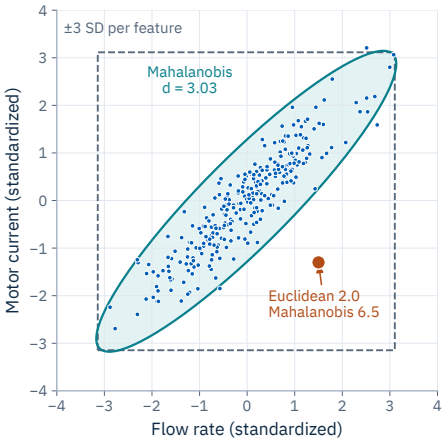
Joint outliers hide inside univariate limits

WHAT COUNTS AS AN ANOMALY

Joint outliers hide inside univariate limits

$$d_M(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)}$$

- Each value of the orange point is within ±3 SD of its feature
- It breaks the flow–current relationship: d_M = 6.5
- For Gaussian data with two features, the 99% cut-off is 3.03
- Estimate μ and Σ robustly so outliers cannot mask themselves



Synthetic correlated features (seed 4): correlation 0.9, 300 rows; covariance estimated from the sample.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 14

Define the symbols: μ is the mean vector, Σ the covariance matrix and p the number of features. The Mahalanobis distance measures distance in units of the data's own spread and correlation.

Flow rate and motor current move together on this pump. The orange point has ordinary values on both axes, so it passes both univariate ±3 SD limits (the grey box). Its combination is impossible under the observed correlation: Euclidean distance from the centre 2.0, Mahalanobis distance 6.5. The teal ellipse is the 99% contour: the square root of the chi-square 99% quantile with 2 degrees of freedom, √9.21 = 3.03.

The chi-square cut-off assumes Gaussian data; treat it as a starting point, not a guarantee. Because the mean and covariance are themselves distorted by outliers, use a robust estimate such as the minimum covariance determinant (sklearn.covariance.MinCovDet; EllipticEnvelope uses it). This is the multivariate version of the masking problem.

Ask: which pairs of energy measurements should move together? Examples: pump speed and flow, RHOB and NPHI within a lithology, power and wind speed along a turbine curve. [3 min]

Reference: https://scikit-learn.org/stable/modules/covariance.html

Physical rules catch what statistics cannot



Statistical detectors learn what is common; physical rules state what is possible. The two are complementary and should be reported separately.

Range rules catch impossible values: negative flow, a neutron porosity above 1, a temperature outside the sensor's rated range. Rate-of-change rules catch spikes and dropouts that no physical process could produce in one sampling interval. Flatline rules catch stuck or frozen sensors, a collective anomaly that most detectors miss. Consistency rules compare related channels: caliper against bit size for borehole washouts, inlet against outlet flow for a pipeline mass balance.

A useful check from the course file: in the rows complete in the six lab curves, the caliper never exceeds the 8.5-inch bit size by more than 0.37 inch, so a one-inch washout rule would flag nothing there. The absence of flags from a physical rule is itself evidence worth recording.

Ask: which rules would you write for an ESP? Examples: frequency within the drive limits, current not zero while running, intake pressure above the pump's minimum. [3 min]

Compute a robust z-score by hand

PRACTICE

5 min · Individual

Compute a robust z-score by hand

Seven intake-pressure readings (bar): 48, 49, 50, 50, 51, 52, 71.

1. The mean is 53.0 and the SD is 8.04. Is 71 beyond |z| = 3?
2. Compute the median, the MAD, and the robust z of 71.
3. Can any reading in a sample of seven exceed |z| = 3?

Give four minutes, then work through the answers on the board.

Answer 1: z = (71 − 53.0) / 8.04 = 2.24. Not flagged. (Sum of squared deviations: 25 + 16 + 9 + 9 + 4 + 1 + 324 = 388; 388 / 6 = 64.67; √64.67 = 8.04.)

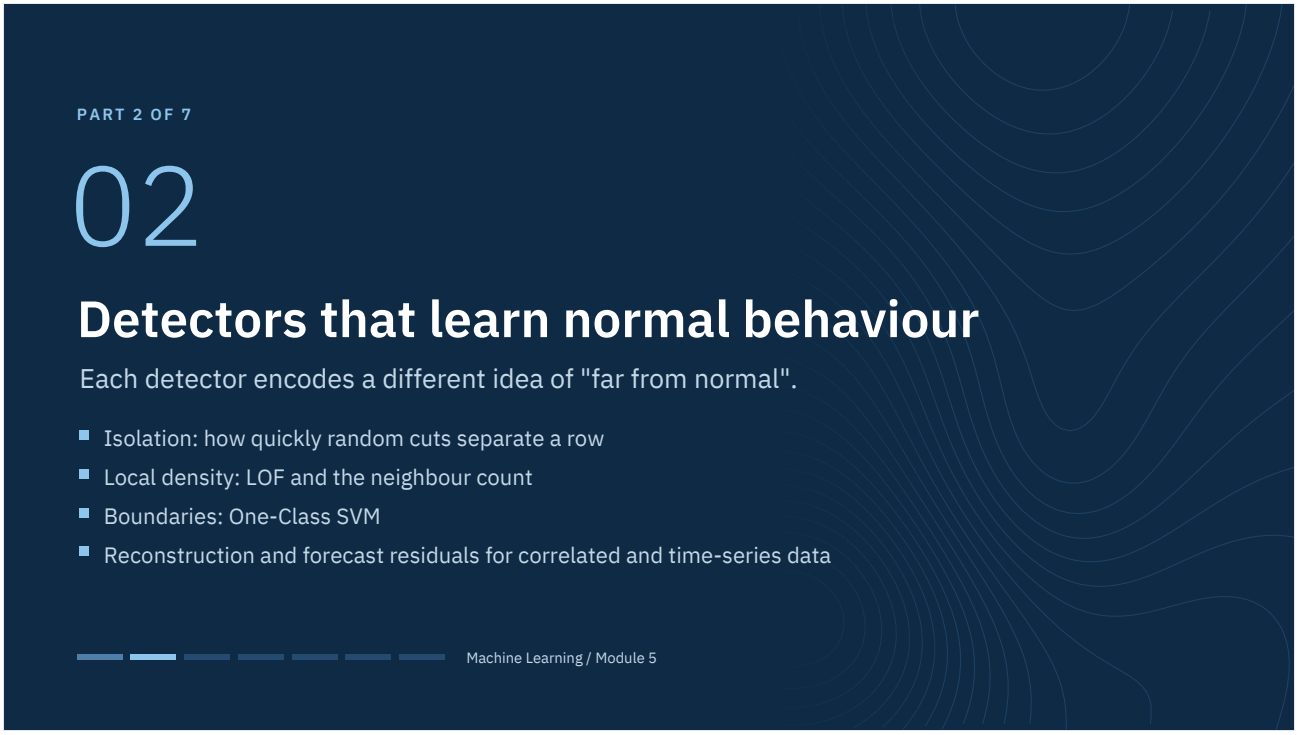
Answer 2: the median is 50. Absolute deviations from 50 are 2, 1, 0, 0, 1, 2, 21; sorted: 0, 0, 1, 1, 2, 2, 21, so the MAD is 1. Robust z = (71 − 50) / (1.4826 × 1) = 14.2. Clearly flagged. The equivalent form 0.6745 × 21 / 1 gives the same 14.2.

Answer 3: no. With the sample standard deviation the largest possible |z| in a sample of n values is (n − 1) / √n, which for n = 7 is 6 / 2.646 = 2.27. With the population SD (ddof = 0, as StandardScaler uses) the bound is √(n − 1) = 2.45. A |z| > 3 rule can never fire on seven readings, however extreme one of them is.

Common errors: using the mean instead of the median when computing the MAD, forgetting to sort, and mixing the 1.4826 and 0.6745 conventions.

Transition: robust rules are the baseline; now we look at detectors that learn a picture of normal from many features. [5 min]

Detectors that learn normal behaviour



This part takes about 50 minutes and ends just before the morning break. It covers the detectors used in both labs plus two ideas that matter for sensor data: reconstruction error and forecast residuals.

Frame each method by its assumption. Isolation Forest assumes anomalies are few and easy to separate with random cuts. LOF assumes that local density matters, so a point can be unusual relative to its neighbours without being globally extreme. One-Class SVM assumes that normal data occupy a compact region in a kernel space. Reconstruction methods assume normal data lie near a low-dimensional subspace. Forecast residuals assume normal behaviour is predictable from context.

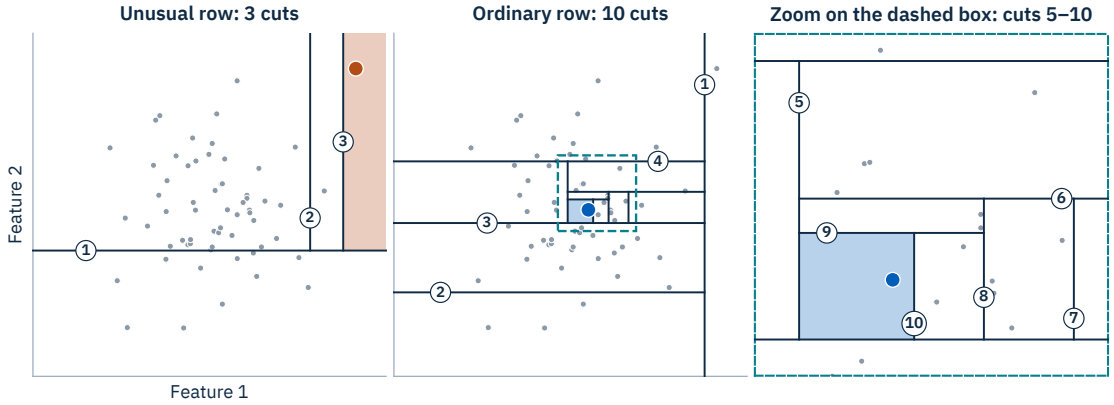
No method is best in general. The right question is which assumption matches the anomaly type and the data you have. [1 min]

Random cuts isolate unusual points in few steps

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Random cuts isolate unusual points in few steps

Each cut: a random feature, and a random value inside the current cell's range.



Synthetic 2-D sample (seed 2): 60 ordinary rows and one unusual row; one random isolation path for each highlighted row.

Explain the mechanism before the formula. An isolation tree repeatedly chooses a feature at random and a split value at random between the minimum and maximum of the rows still in the current cell. It keeps cutting until the row of interest sits alone.

Left: the unusual row in the upper right is alone after three cuts, because almost any random cut near the edge of the data separates it. Middle: an ordinary row in the dense centre needs ten cuts; the first four shrink the region and the zoom on the right shows cuts 5 to 10 closing in.

One random path is noisy, which is why the method builds a forest of many trees and averages the path length (next slide).

The idea is attractive for tabular sensor data: no distance metric to choose, no density to estimate, and it scales well. Its weakness follows from the same idea: random axis-parallel cuts do not know about time or operating context, so contextual anomalies need context features. [3 min]

Isolation Forest turns short paths into high scores

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Isolation Forest turns short paths into high scores

$$s(x, n) = 2^{-E[h(x)]/c(n)}$$

- Average the path length $h(x)$ over many random trees
- $c(n)$ normalizes by the average path for n rows
- s near 1: isolated quickly; well below 0.5: ordinary
- Investigate ranked rows; validate any alert threshold



c(61) = 7.38 for the 61-row sample; 1,000 random paths per highlighted row.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 19

Define the terms: h(x) is the number of cuts needed to isolate x in one tree, E[h(x)] its average over trees, and c(n) the average path length of an unsuccessful search in a binary tree built on n rows, used to normalize (Liu, Ting and Zhou, 2008).

On our 61-row example the unusual row needs 3.14 cuts on average, giving s = 2^(−3.14 / 7.38) = 0.74; the ordinary row needs 10.04 cuts, giving s = 2^(−10.04 / 7.38) = 0.39.

The method does not require a parametric density model and is a good tabular baseline. It can still miss contextual anomalies and it responds to uninformative features, which dilute the random cuts. In practice each tree uses a subsample (256 rows by default in scikit-learn) and a depth limit.

Software score signs vary: scikit-learn's score_samples returns the opposite of s, so lower means more abnormal, and decision_function subtracts an offset set by the contamination parameter. Read the API before thresholding. [3 min]

Reference: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

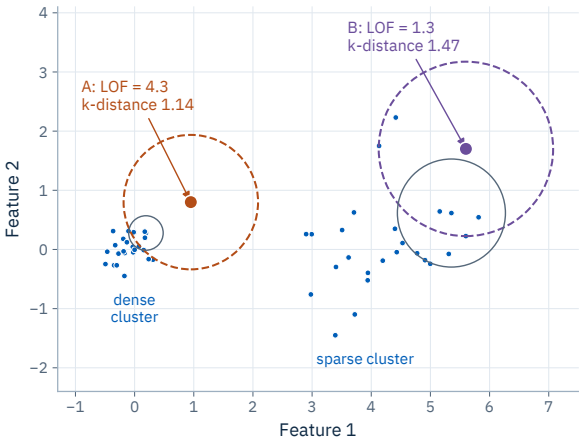
Local Outlier Factor compares neighbourhoods

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Local Outlier Factor compares neighbourhoods

$$\mathrm{LOF}_k(p) = \frac{1}{k} \sum_{o \in N_k(p)} \frac{\mathrm{lrd}_k(o)}{\mathrm{lrd}_k(p)}$$

- lrd: inverse of the mean reachability distance to the k neighbours
- LOF near 1: as dense as its neighbours; well above 1: sparser
- A is closer to its neighbours than B, yet A is the local outlier
- Scaling and the neighbour count change the result



Synthetic clusters (seed 8); LocalOutlierFactor with n_neighbors = 6. Dashed circles: k-distances of A and B; grey circles: one neighbour of each.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 20

Define: N_k(p) is the set of the k nearest neighbours of p. The reachability distance from p to o is the larger of d(p, o) and the k-distance of o, which smooths out tiny distances inside dense clusters. The local reachability density lrd is the inverse of the mean reachability distance. LOF compares the density of p with the densities of its neighbours (Breunig et al., 2000).

Read the figure: A has k-distance 1.14 while its six neighbours inside the dense cluster have k-distances between 0.21 and 0.29, so A is much sparser than its neighbourhood: LOF = 4.3. B is farther from its neighbours (k-distance 1.47), but so are they (0.80 to 1.88), so LOF = 1.3. An observation can be unusual locally despite not being globally extreme.

Scaling and the neighbour count influence the result. In scikit-learn, default LOF performs outlier detection on its training data (fit_predict and negative_outlier_factor_). For novelty detection, initialize novelty=True and use predict, decision_function or score_samples on new observations only, never on the training rows as if they were new. This distinction often causes lab errors. [3 min]

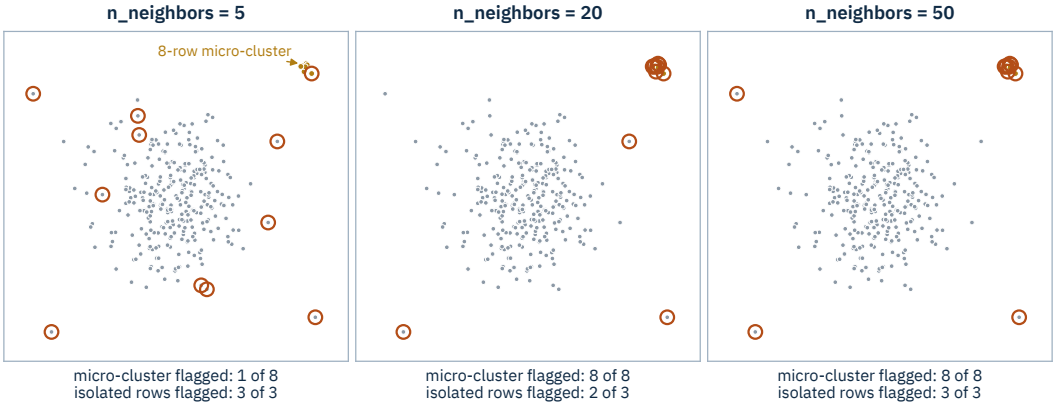
Reference: https://scikit-learn.org/stable/modules/outlier_detection.html

The neighbour count decides what counts as local

DETECTORS THAT LEARN NORMAL BEHAVIOUR

The neighbour count decides what counts as local

Set n_neighbors above the size of any group of rows that should still count as unusual.



Synthetic data (seed 12): 300 ordinary rows, an 8-row micro-cluster, and 3 isolated rows; rings mark the 11 highest LOF scores.

With 5 neighbours the eight rows of the micro-cluster are each other's neighbours, so they look perfectly dense and only one of them reaches the top 11; the rings go to edge points of the main cloud instead. With 20 or 50 neighbours the neighbourhoods reach the main cloud, the micro-cluster's density is compared with it, and all eight rows are flagged.

In sensor data a micro-cluster is often exactly what you want to find: a stuck value repeated for a few hours, or a short excursion into an unusual operating state. Think of n_neighbors as the dividing line between a group and a cluster: groups smaller than it can stand out as outliers, groups larger than it act as clusters of their own. Breunig et al. (2000) discuss choosing a range of values for this reason. scikit-learn's default is 20; try a few values and compare which rows are flagged.

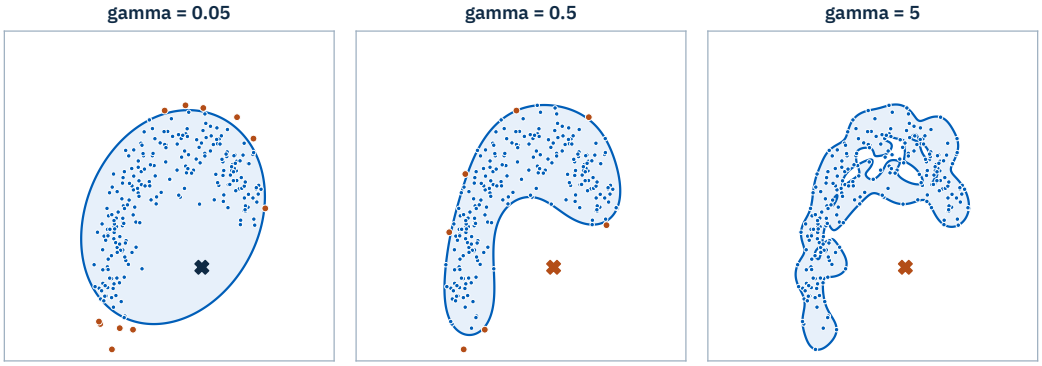
Lab link: in Lab 2 you will sweep n_neighbors from 5 to 100 with contamination fixed at 0.05. The number of flagged rows cannot change, because contamination fixes it; compare which rows are flagged instead. [3 min]

One-Class SVM estimates a boundary

DETECTORS THAT LEARN NORMAL BEHAVIOUR

One-Class SVM estimates a boundary

nu caps the share of training rows outside; gamma sets how tightly it wraps.



new ordinary rows flagged: 7.9%
interior probe (X) flagged: no

new ordinary rows flagged: 8.6%
interior probe (X) flagged: yes

new ordinary rows flagged: 23.8%
interior probe (X) flagged: yes

Synthetic arc-shaped reference data (seed 6, standardized); RBF kernel, nu = 0.05. Flag rates measured on 2,000 new rows from the same generator.

One-Class SVM fits a boundary around a reference population in a kernel space (Schölkopf et al., 2001). With the RBF kernel, gamma sets the locality: small gamma gives a smooth boundary, large gamma one that wraps individual rows. Scale features first, because the kernel uses distances.

Left, gamma = 0.05: the boundary is an ellipse that includes the empty interior of the arc, so a probe placed there is not flagged. Middle, gamma = 0.5: the boundary follows the arc, 8.6% of new ordinary rows are flagged and the probe is caught. Right, gamma = 5: a jagged boundary with holes flags 23.8% of new ordinary rows: an overfitted picture of normal.

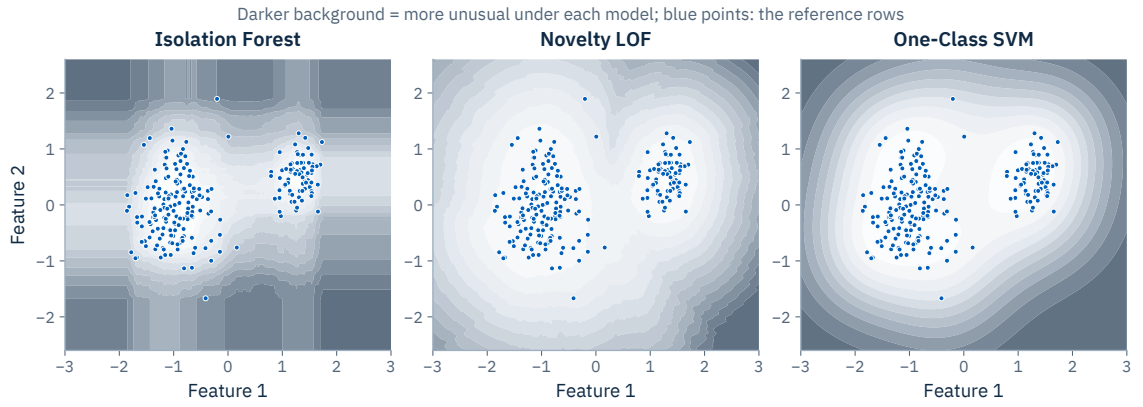
nu is an upper bound on the fraction of training rows outside the boundary and a lower bound on the fraction of support vectors. It is not a calibrated estimate of real fault prevalence. Many support vectors sit almost exactly on the boundary, so scores of about zero are neither clearly flagged nor clearly safe.

Check sensitivity to contaminated training data and to dataset size; training time grows quickly with the number of rows, which is why the lab notebook subsamples above 10,000 rows. [3 min]

Methods rank unusual observations differently

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Methods rank unusual observations differently



Three detectors trained on the same synthetic reference set (seed 42). Backgrounds show model scores, not fault probabilities.

The blue points are the reference rows each model was trained on; the shading is each model's score, darker meaning more unusual. Point out how local density and global boundary assumptions differ. Isolation Forest produces axis-aligned, blocky score regions because its cuts are axis-parallel. Novelty LOF produces a score landscape that follows the local density of each cluster, including the valley between them. One-Class SVM produces a smooth boundary around both clusters.

Ask which model might flag a valid low-density operating regime. Answer: LOF and One-Class SVM would both treat a rarely visited but valid regime as unusual if it is sparse in the reference data; Isolation Forest would too if it is isolated. None of them knows which regimes are valid; that knowledge must come from engineers or from context features.

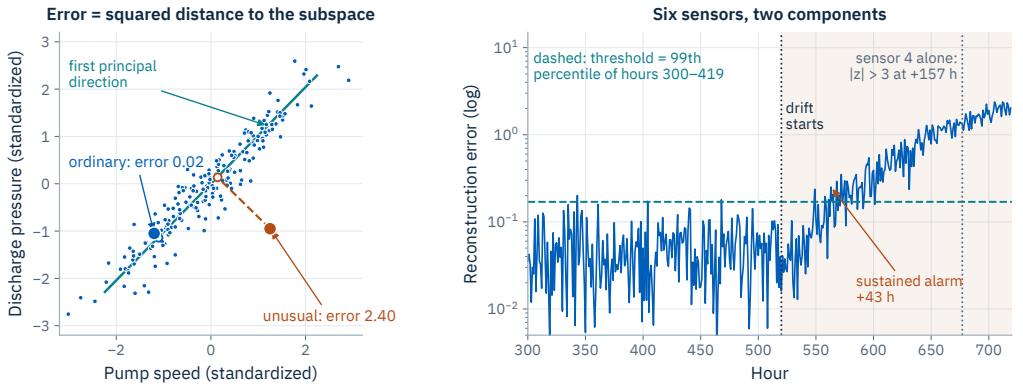
Do not compare raw score magnitudes across methods; their scales and signs differ. Compare rankings, or convert each method's scores to ranks or percentiles before combining them. [2 min]

Reconstruction error flags broken relationships

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Reconstruction error flags broken relationships

PCA (Module 4) keeps the correlations; rows that break them reconstruct badly.



Left: synthetic correlated pair (seed 13). Right: six synthetic channels (seed 32); PCA fitted on hours 0–299, threshold from hours 300–419, sensor 4 drifts from hour 520.

Recall PCA from Module 4. Project each row onto the first few principal components and map it back; the squared distance between the row and its reconstruction is the reconstruction error, sometimes called the squared prediction error or Q statistic.

Left: two strongly correlated sensors. The ordinary row sits close to the first principal direction (error 0.02); the unusual row has plausible values on both axes but breaks the relationship (error 2.40).

Right: six channels driven by two hidden factors, speed and load. PCA with two components is fitted on hours 0 to 299, and the threshold is the 99th percentile of errors on hours 300 to 419, which were not used for fitting. From hour 520 sensor 4 drifts slowly. A sustained alarm (three consecutive hours above the threshold) comes 43 hours after the drift starts, while sensor 4 on its own first exceeds |z| = 3 only after 157 hours. Note the isolated crossings before the drift: a 99th-percentile threshold produces about one false crossing per hundred hours by construction.

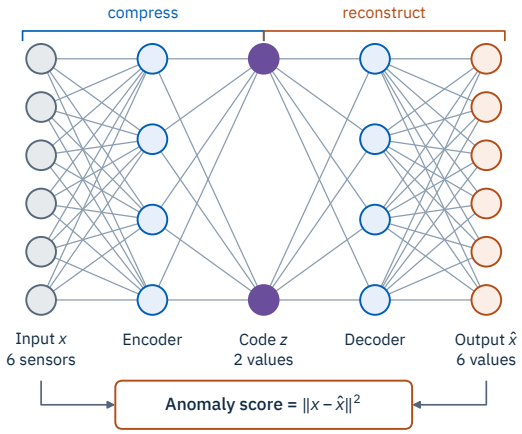
Fit the scaler, the PCA and the threshold on reference data only, and refit them when the process changes. [3 min]

Autoencoders learn nonlinear reconstructions

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Autoencoders learn nonlinear reconstructions

- The encoder compresses; the decoder rebuilds the input
- Train on normal operation only; score by reconstruction error
- A linear autoencoder with squared error spans the PCA subspace
- Needs more data and tuning: see the neural network course



Schematic undercomplete autoencoder (6-4-2-4-6 units).

Machine Learning / Module 5 · Rare events, anomalies, and model selection 25

An autoencoder is a neural network trained to reproduce its input through a narrow bottleneck. Trained on normal operation only, it learns to rebuild normal patterns well and unusual patterns badly, so the reconstruction error becomes an anomaly score, exactly as with PCA.

The link to PCA is precise: an autoencoder with linear activations trained with squared error learns the same subspace as PCA with the same number of components (Baldi and Hornik, 1989). Nonlinear activations let it follow curved relationships, such as pump curves, at the cost of more data, more tuning and more ways to overfit. A network that is too flexible can learn to reconstruct anomalies too.

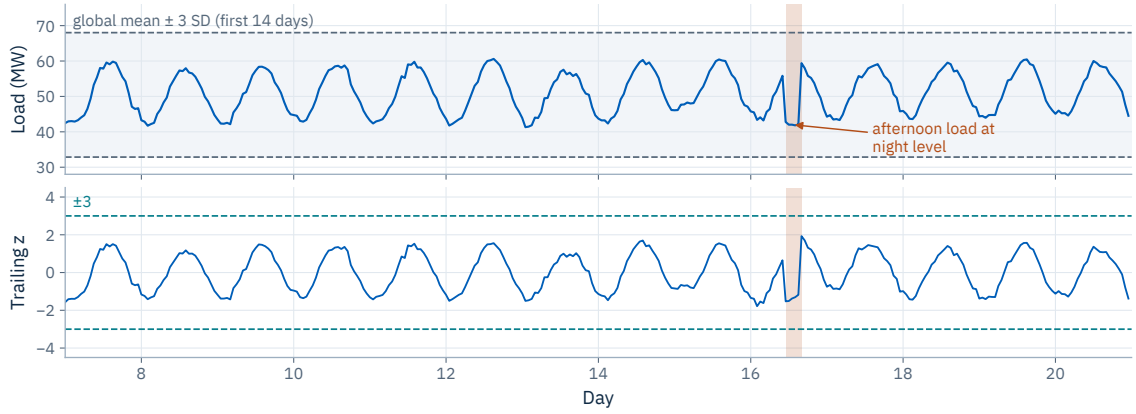
The same evaluation rules apply: fit on reference data, set the threshold on separate validation data, and review what is flagged. The neural network course covers autoencoders on Day 2, convolutional autoencoders on Day 4, and sequence anomaly detection on Day 5.

Ask: when would you still prefer PCA? Answer: small data, a need for transparency (you can inspect loadings and per-channel residuals), and when a linear model already reconstructs normal data well. [2 min]

Global limits and rolling z-scores miss daily context

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Global limits and rolling z-scores miss daily context



Synthetic hourly load (seed 17); five afternoon hours near day 16.5 set to night level. Top: mean ± 3 SD of the first 14 days. Bottom: z-score against the trailing 24 hours.

Many energy series have strong daily and weekly cycles: electrical load, solar output, pipeline flow, even pump current. A contextual anomaly here is five afternoon hours near day 16.5 that sit at night-time level, about 42 MW instead of about 58 MW.

Top: the global band from the first 14 days runs from 32.9 to 68.0 MW, so the anomaly is well inside it. The band is wide precisely because it contains the whole daily cycle.

Bottom: a trailing 24-hour z-score compares each hour with the mean and SD of the previous 24 hours. The window again contains a full cycle, so its SD is large, and the anomaly only reaches z = −1.5; the largest |z| anywhere else is 1.9. Nothing crosses ±3.

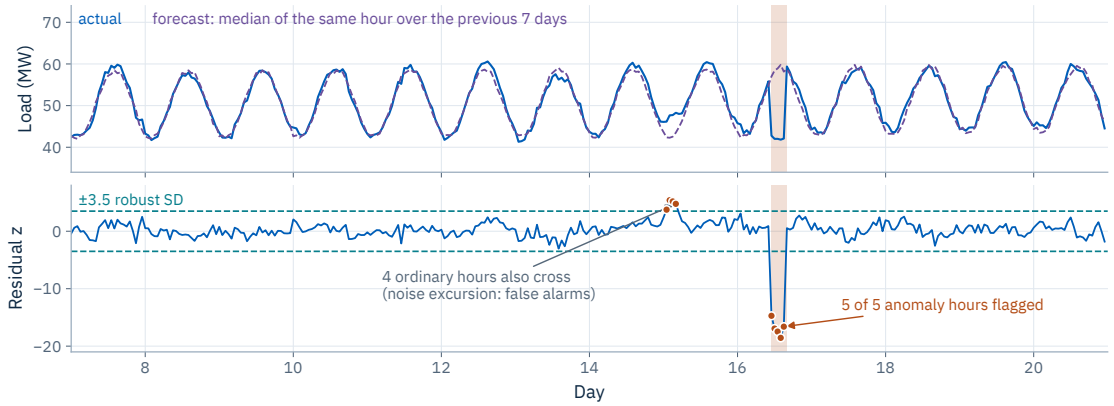
Seasonality pitfalls to name: windows that mix different phases of a cycle, weekly and holiday effects, operating-mode changes that shift the level, and baselines that absorb the anomaly itself. The fix is to compare like with like, which is what a seasonal forecast does (next slide). [3 min]

Forecast residuals expose contextual anomalies

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Forecast residuals expose contextual anomalies

Compare each hour with what that hour usually looks like, then threshold the residual.



Same series. Forecast: median of the same hour over the previous 7 days; residuals scaled by the median and MAD of days 7–14; threshold 3.5.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 27

The forecast for each hour is the median of the same hour over the previous seven days, a seasonal naive forecast. The residual (actual minus forecast) removes the daily cycle. Scaled by a robust spread estimated on days 7 to 14, the five anomalous hours have residual z-scores between −14.7 and −18.6 and are all flagged.

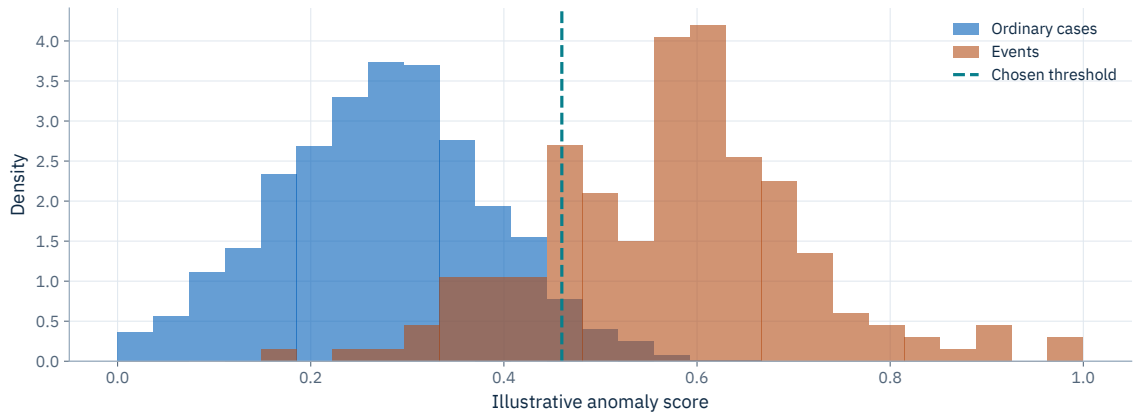
Two honest details. First, the median rather than the mean keeps one anomalous day from contaminating the next week's forecasts. Second, four ordinary hours at the start of day 15 also cross the threshold because of a noise excursion. Any threshold has a false-alarm rate, and a persistence rule (for example, three of four hours above the threshold) is one way to suppress short excursions; we return to alert rules in Part 3.

Better forecasts give cleaner residuals: add weekly seasonality, temperature for load, or operating mode for pumps. The forecasting models themselves, including sequence neural networks, are the subject of Day 5 of the neural network course. [3 min]

A score and an alert are separate outputs

DETECTORS THAT LEARN NORMAL BEHAVIOUR

A score and an alert are separate outputs



Simulated score distributions (seed 42), not a fitted detector, with a chosen threshold. The overlap creates both missed events and false alarms.

The distributions are simulated to teach thresholding, not fitted field evidence. Every detector we have seen outputs a continuous score; an alert is a separate decision made by comparing the score with a threshold, often followed by rules such as persistence and cooldown.

Ask learners to move the threshold mentally to the left: more events are caught, and more ordinary cases are alerted. Moving it right does the opposite. There is no threshold that removes both kinds of error when the distributions overlap.

An unsupervised score needs labelled review or an operational constraint to set a defensible threshold: either reviewed events that allow precision and recall to be estimated, or a review capacity (for example, the team can investigate five alerts a week). The contamination parameter is not such a constraint; it simply moves the threshold to a chosen quantile of the training scores. [2 min]

Score signs differ across scikit-learn detectors

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Score signs differ across scikit-learn detectors

Estimator	Lower score_samples means	predict for outliers	Watch for
IsolationForest	More abnormal	−1	decision_function subtracts offset_
LocalOutlierFactor	More abnormal	−1	Only with novelty=True; else fit_predict
OneClassSVM	More abnormal	−1	Scale features; tune nu and gamma
EllipticEnvelope	More abnormal	−1	Assumes one elliptical cloud

Convert to "higher = more unusual" once, in one place, and test it.

All four scikit-learn outlier detectors follow the same conventions: predict returns −1 for outliers and +1 for inliers, score_samples is lower for more abnormal rows, and decision_function equals score_samples minus an offset, so that negative values are outliers under the fitted threshold.

The traps are elsewhere. The anomaly score in the Isolation Forest paper is high for anomalies, while score_samples returns its opposite. LOF in its default outlier mode offers only fit_predict and the attribute negative_outlier_factor_ for the training rows; score_samples, decision_function and predict exist only with novelty=True. EllipticEnvelope's score_samples is the negative squared Mahalanobis distance under a robust covariance.

Practical rule: write one small function that converts each detector to "higher means more unusual", test it on a known outlier, and use it everywhere, including in plots. Verify the distinction between score_samples, decision_function and predict for each estimator before thresholding. [2 min]

Reference: https://scikit-learn.org/stable/modules/outlier_detection.html

Combine detectors by rank, not by raw score

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Combine detectors by rank, not by raw score

```
from scipy.stats import rankdata

scores = {                          # higher = more unusual
    "iforest": -iso.score_samples(X),
    "lof": -lof.negative_outlier_factor_,
    "ocsvm": -ocsvm.decision_function(X),
}
pct = {k: rankdata(v) / len(v)   # 0-1 percentile ranks
       for k, v in scores.items()}
combined = np.mean(list(pct.values()), axis=0)
review = np.argsort(-combined)[:50]   # top 50 to review
```

- Raw scores differ in scale and sign across methods
- Percentile ranks put them on one 0–1 scale
- The average rank is a review order, not a probability
- Keep each method's rank to explain why a row was flagged

Machine Learning / Module 5 · Rare events, anomalies, and model selection 30

Part 6 of the Lab 2 notebook ends with this review order, done more carefully than a vote. In the code, the three detectors were fitted in outlier mode on the same standardized matrix X: iso is a fitted IsolationForest, lof a LocalOutlierFactor after fit_predict, and ocsvm a fitted OneClassSVM. The notebook ranks the three score columns stored in Parts 3 to 5, which its unusualness helper has already oriented as "higher = more unusual", so it gives the same order as this code.

Each score is first converted to "higher = more unusual", using the conventions from the previous slide. rankdata then replaces each score with its rank, and dividing by the number of rows gives a percentile between 0 and 1, so no method dominates because of its units. The mean percentile gives one review order.

Compared with counting votes above each method's threshold, rank averaging does not depend on three separate contamination settings and it separates rows that all methods rank highly from rows that just pass every threshold.

Limits: the combined rank is still not a probability of a fault, and methods with shared assumptions (IQR and z-score, for instance) effectively get double weight; combine methods that look at the data differently. [3 min]

Fit the detector on a reference period only

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Fit the detector on a reference period only

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

ref = df[df["time"] < cutoff]      # reviewed, ordinary period
new = df[df["time"] >= cutoff]

det = make_pipeline(StandardScaler(),
                    IsolationForest(n_estimators=300, random_state=0))
det.fit(ref[features])

s_ref = -det.score_samples(ref[features])   # higher = more unusual
threshold = np.quantile(s_ref, 0.995)        # a start; validate it
alerts = -det.score_samples(new[features]) > threshold
```

Machine Learning / Module 5 · Rare events, anomalies, and model selection 31

Walk through the code line by line. df is a table with a time column and the feature columns listed in features; cutoff is the end of a period that engineers reviewed and accepted as ordinary.

The pipeline makes sure the scaler learns its mean and SD from the reference period only, just as preprocessing had to stay inside training folds in Module 1. Pipeline exposes score_samples when the last step does. The minus sign converts scikit-learn's convention (lower = more abnormal) to "higher = more unusual".

The threshold is the 99.5th percentile of the reference scores, so about 0.5% of reference rows would alert. That only predicts the false-alert rate on new data if the reference is clean and the process is stationary; it says nothing about recall. Treat it as a starting point, then validate it against reviewed events or an agreed review capacity, and freeze it before any final test.

Common error: fitting the detector on all data, including the period you later evaluate, which makes future anomalies part of "normal". [3 min]

Contamination is not a discovered prevalence

PRACTICE

4 min Pairs

Contamination is not a discovered prevalence

An analyst sets contamination = 0.02 and reports that 2% of the equipment is faulty.

1. What did the parameter actually control?
2. What evidence is missing from the conclusion?

Machine Learning / Module 5 · Rare events, anomalies, and model selection 32

Answer 1: contamination generally controls the threshold, or the expected fraction of training rows treated as outliers, depending on the estimator. In IsolationForest it sets offset_ so that 2% of the training rows get a negative decision_function; the ranking itself does not change. It does not reveal fault prevalence: 2% of rows are flagged because 2% was requested.

Answer 2: the conclusion needs validated labels or reviewed events, an appropriate sampling of what was reviewed (including unflagged rows, to estimate what was missed), a definition of "faulty" at the equipment level rather than the row level, and an understanding of the method's ranking quality. It also needs the unit right: 2% of rows is not 2% of equipment.

You will see this directly in Lab 1: Isolation Forest flags exactly 1%, 5% and 10% of the rows as contamination goes from 0.01 to 0.10.

Common misconception: "contamination=auto discovers the rate". In scikit-learn, auto uses the fixed offset from the original paper; it is still an assumption. [4 min]

Choose a first detector by its assumptions

DETECTORS THAT LEARN NORMAL BEHAVIOUR

Choose a first detector by its assumptions

Method	Assumes	Good first use	Watch for
Robust z / IQR	One population per feature	Single-sensor quality control	Mixtures, context
Mahalanobis	One elliptical cloud	Correlated channels	Masking: fit robustly
Isolation Forest	Anomalies few and different	Tabular baseline	Irrelevant features
LOF	Local density matters	Clusters of different density	n_neighbors, scaling
One-Class SVM	Compact normal region	Clean reference, modest size	gamma, nu, runtime
PCA reconstruction	Normal rows near a subspace	Many correlated sensors	Nonlinear relationships

Machine Learning / Module 5 · Rare events, anomalies, and model selection 33

Use this table as the summary of Parts 1 and 2 and as a checklist for the labs. Read it by rows: each method is a hypothesis about what normal looks like.

Suggested order in a new project: plot the data; write physical rules; apply robust univariate rules per regime; then one multivariate detector that matches the data (Isolation Forest for mixed tabular features, PCA reconstruction for many correlated sensors, LOF when regimes differ in density). Add a second method only to answer a specific question, not to build a larger vote.

Ask: which method would you try first for the six-channel pump example? Answer: PCA reconstruction or Mahalanobis, because the channels are strongly correlated and the anomaly is a broken relationship. For the well-log file? Robust rules per zone plus Isolation Forest, which is what Lab 1 does.

Transition: after the break we apply the first three rows of this table to the course well-log file. [3 min]

Lab 1: statistical rules and Isolation Forest

LABORATORY

65 min

Lab 1: statistical rules and Isolation Forest

- ☐ Part 1: load, plot one well, drop incomplete rows; record the rows and wells lost
- ☐ Part 2: flag |z| > 3 and IQR outliers; compare the two sets
- ☐ Part 3: fit Isolation Forest (contamination 0.05) and plot its score track
- ☐ Sweep contamination from 0.01 to 0.10 and explain what changes

NOTEBOOK

`exercises/anomaly_detection/Anomaly_Detection_Student.ipynb`

EVIDENCE TO SUBMIT

- Row and well counts before and after cleaning
- z-score versus IQR overlap counts, with one reviewed interval
- One sentence on what contamination controls

Machine Learning / Module 5 · Rare events, anomalies, and model selection 34

Timing: 5 minutes of briefing, 50 minutes of work, 10 minutes of debrief. The notebook works on well_logs_data.csv (five wells named 15/9-F-..., as supplied in the course folder) and uses GR, RHOB, NPHI, DT, CALI and BS.

Setup: seaborn is in requirements-labs.txt (plot style, heatmaps in Parts 4 and 6). The notebook opens with its protocol: an exploratory review of one fixed table, with every rule, scaler and detector fitted on the complete rows it flags; there are no labels, so it reports flags and rankings, never precision or recall. A helper, unusualness, converts every score once to "higher = more unusual", and a check confirms that the flagged rows are the highest-scoring ones.

Reference results (identical with scikit-learn 1.5, 1.7 and 1.9): 27,845 rows load; dropping rows with missing values in the six curves leaves 5,493 rows from three wells. The Part 1 audit table shows why: 15/9-F-1 C and 15/9-F-11 B disappear entirely, because DT is missing in all of their rows. The notebook keeps this cleaning on purpose and states the consequence under the audit. The |z| > 3 rule flags 149 rows (2.71%) and the IQR rule 522 rows (9.50%); every z-score row is also an IQR row. The review cell finds the thickest IQR-flagged interval in 15/9-F-1 A at 3,620.8–3,627.5 m MD: 68 rows, flagged on GR alone (median 113 API, VSH 0.83), so shale, not a bad measurement. Isolation Forest flags 55, 275 and 550 rows at contamination 0.01, 0.05 and 0.10, with thresholds of 0.600, 0.536 and 0.512 on its score (the paper's s); the notebook confirms that the flagged sets are nested.

Common errors: plotting depth without inverting the y-axis; forgetting .copy() after filtering and getting pandas warnings; storing boolean masks without converting to 0/1; using df instead of df_clean for later parts; flipping a score sign by hand instead of using the helper.

Circulate and ask each pair: "what did cleaning remove, and does it matter for what we can conclude?" [65 min]

Cleaning silently removed two of five wells



Start the debrief with the data audit, not with the detectors. Dropping incomplete rows kept 5,493 of 27,845 rows, about 20%. Two of the five wells, 15/9-F-1 C and 15/9-F-11 B, have no DT at all, so every one of their rows was removed. dropna gives no warning; only the audit table in Part 1 of the notebook (rows per well before and after, and the share of each curve missing in each well) makes the loss visible. Any conclusion about "anomalies in the field data" is really about three wells.

BS (bit size) is 8.5 inches in every remaining row. StandardScaler sets a zero-variance feature's scale to 1, so its standardized value is 0 everywhere; it contributes nothing to any detector, and its IQR fences are 8.5 to 8.5.

Ask the room what they would do differently. Good answers: analyse wells separately or drop DT and keep all five wells; report results per well; remove constant features explicitly; record every cleaning step in the notebook's first cell, as in the prediction contract of Module 1.

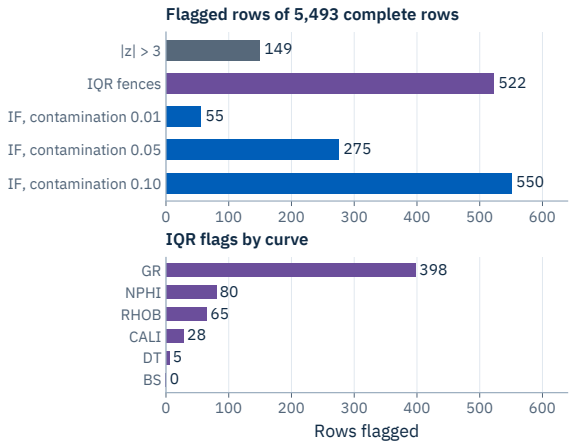
This is the same lesson as leakage in Module 1 seen from the other side: silent data handling decides what the evaluation means. [5 min]

Rules disagree, and contamination fixes the count

DISCUSSION

Rules disagree, and contamination fixes the count

- Every |z| > 3 row is also an IQR row; IQR adds 373 more
- 398 IQR flags are high GR: shale-rich rows (median VSH 0.79)
- Isolation Forest flags exactly 1%, 5%, and 10% of the rows
- A higher contamination only lowers the threshold on one ranking



Computed from well_logs_data.csv (course data): 5,493 complete rows; IsolationForest(n_estimators=100, random_state=42).

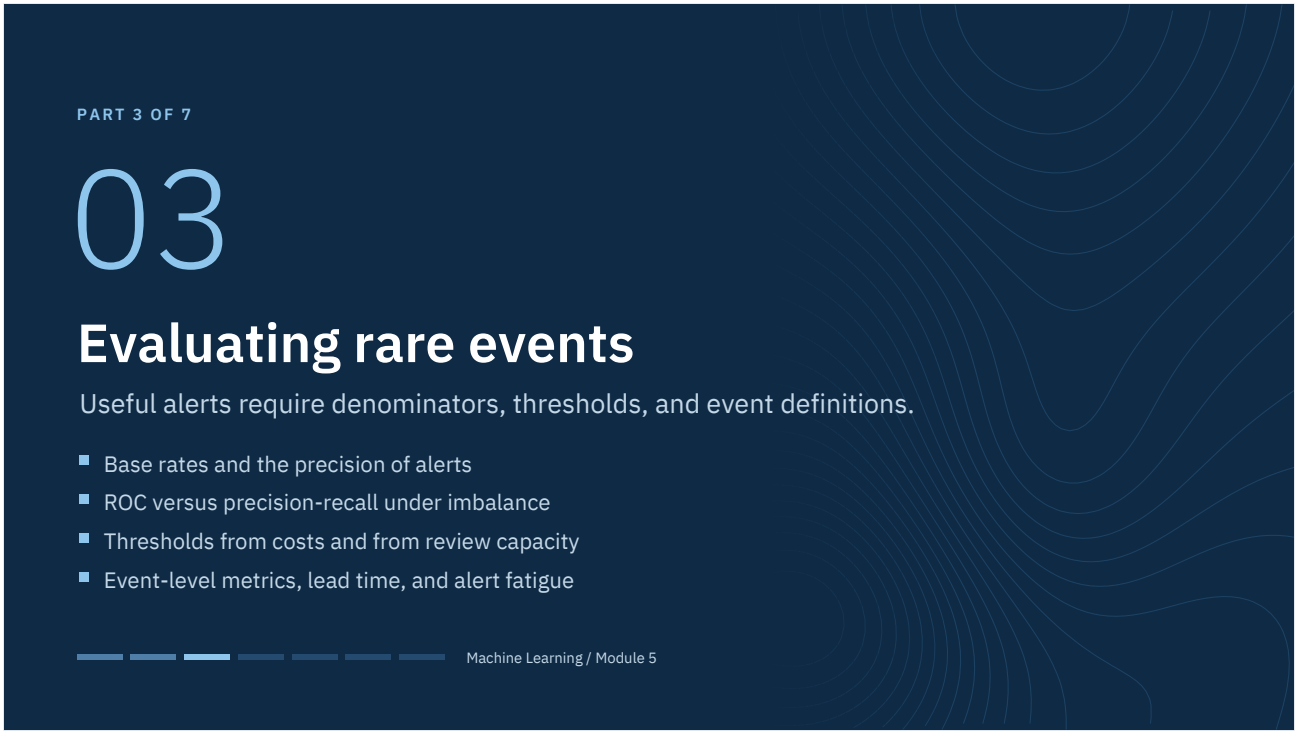
Machine Learning / Module 5 · Rare events, anomalies, and model selection 36

Top panel: the IQR rule flags 522 rows against 149 for |z| > 3, and all 149 z-score rows are inside the IQR set. Bottom panel: 398 of the IQR flags come from gamma ray, all on the high side. Those rows have a median shale volume (VSH, supplied in the file) of 0.79 against 0.28 overall: they are shale-rich intervals, exactly the mixture effect of the fences slide in Part 1. They are unusual in a sand-dominated sample, not bad measurements. The notebook's review cell prints these numbers and the thickest flagged interval, 68 rows at 3,620.8–3,627.5 m in 15/9-F-1 A, flagged on GR alone.

Isolation Forest flags 55, 275 and 550 rows: 1%, 5% and 10% of 5,493, to within one row. Because the forest is the same (same random_state) and contamination only moves the threshold, the three flagged sets are nested: every row flagged at 0.01 is flagged at 0.05 and at 0.10. The notebook prints the three thresholds on its "higher = more unusual" score (0.600, 0.536 and 0.512) and checks the nesting.

Take answers to the deliverable sentence. A good one: "contamination sets the fraction of rows flagged; it is our assumption, not a finding." Then ask what would be needed to choose it: reviewed intervals, or the number of intervals a petrophysicist can review. [5 min]

Evaluating rare events



This part takes about 40 minutes and runs until lunch. We move from finding unusual rows to judging a detector, whether supervised or unsupervised, once some events are known.

Ask learners to recall precision and recall from Module 1: precision is the share of alerts that are real events, recall the share of events that raise an alert. With rare events, a false-positive rate that sounds small can still generate many false alarms, because it multiplies a very large number of ordinary rows.

The second half of the part changes the unit of analysis from rows to events. Operators do not experience rows; they experience trips, alarms, and the hours of warning between them. [1 min]

Prevalence changes the meaning of a false alarm rate

PRACTICE

4 min Individual

Prevalence changes the meaning of a false alarm rate

In 10,000 observations, 100 are events. A detector has 80% recall and a 5% false-positive rate.

1. How many true alerts and false alerts occur?
2. What fraction of alerts are true events?

Let learners work alone for two minutes, then compare with a neighbour.

Answer 1: true positives = 0.80 × 100 = 80. False positives = 0.05 × 9,900 = 495.

Answer 2: precision = 80 / (80 + 495) = 80 / 575, about 13.9%. Round only after calculating the counts.

Many people guess about 80% or 95%, because both headline rates sound good. The trap is that the 5% applies to a group 99 times larger than the event group.

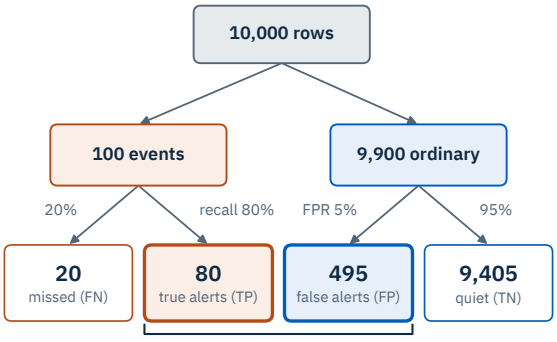
State the assumptions: the rates apply to this population, and the example counts independent observation-level events. In a time series, consecutive rows are not independent and events span many rows, which is why we will soon count events rather than rows. [4 min]

Most alerts are false when events are rare

DISCUSSION

Most alerts are false when events are rare

- 80 true alerts and 495 false alerts: precision 13.9%
- False alerts outnumber true ones about six to one
- The same detector at 10% prevalence: precision 64%
- Quote precision at the prevalence of deployment



Alerts: 80 + 495 = 575

Precision = 80 / 575 = 13.9%

Exact arithmetic for 10,000 rows, 1% prevalence, recall 80%, false-positive rate 5%.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 39

Draw the tree on the board as you explain it: split the population into events and ordinary rows first, then apply recall and the false-positive rate within each branch. This natural-frequency format is much easier to reason with than conditional probabilities.

Of 575 alerts, 80 are true: 495 / 80 is about 6.2 false alerts per true one. At 10% prevalence the same detector would see 1,000 events and 9,000 ordinary rows: 800 true alerts and 450 false ones, precision 800 / 1,250 = 64%. Nothing about the detector changed, only the base rate.

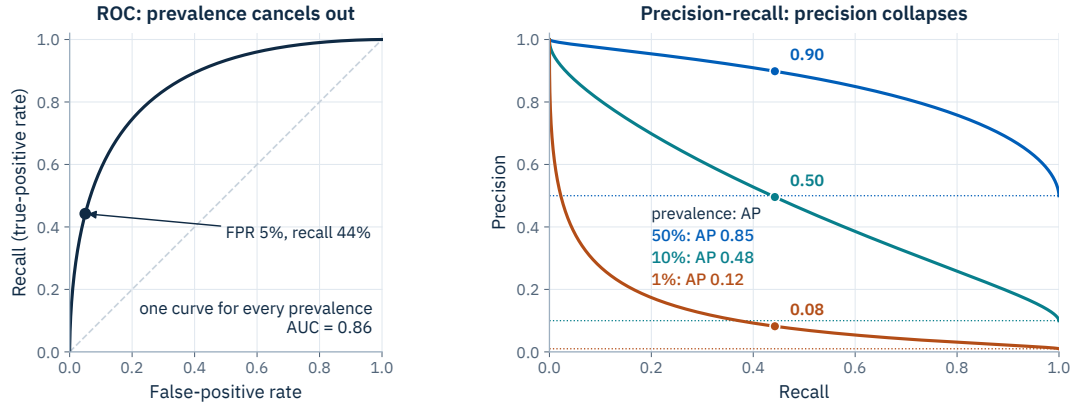
Consequences for reports: never quote precision without the prevalence it was measured at; make sure the validation and test sets keep the prevalence of deployment (do not rebalance them); and when prevalence differs between sites or seasons, expect precision to differ even if the detector is identical. [3 min]

ROC curves hide what prevalence does to precision

EVALUATING RARE EVENTS

ROC curves hide what prevalence does to precision

One detector, three prevalences: one ROC curve, three precision-recall curves.



Exact binormal scores: ordinary N(0, 1), events N(1.5, 1). Dots mark the threshold with a 5% false-positive rate (recall 0.44) and the resulting precision.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 40

The scores are exact binormal distributions, so there is no sampling noise. Left: the ROC curve plots recall against the false-positive rate. Both are computed within one class, so prevalence cancels out: the same curve (AUC 0.86) holds at any event rate.

Right: precision mixes the two classes, so it depends on prevalence. At the threshold with a 5% false-positive rate, recall is 0.44 for all three, but precision is 0.90 at 50% prevalence, 0.50 at 10% and 0.08 at 1%. Average precision (AP) falls from 0.85 to 0.48 to 0.12. The dotted lines show that a random ranking has precision equal to the prevalence.

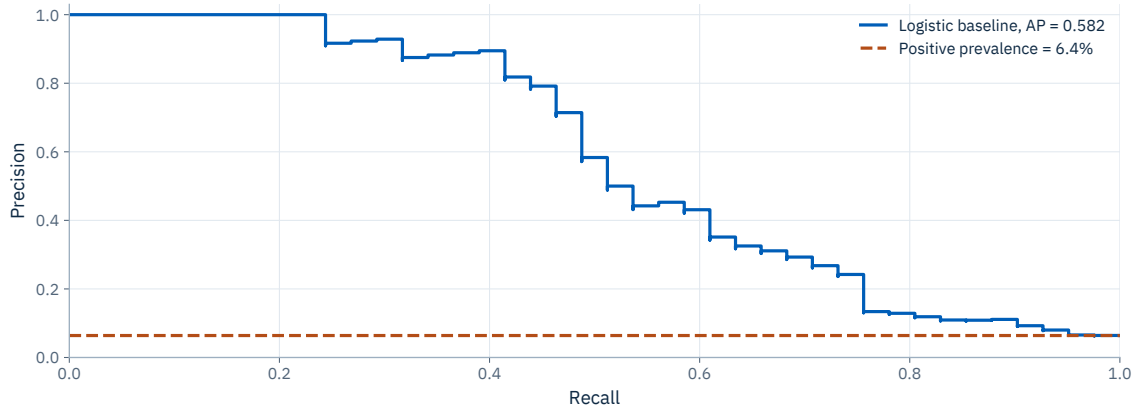
This is why precision-recall is more informative for rare events (Saito and Rehmsmeier, 2015). ROC-AUC is still useful for comparing rankings at a fixed prevalence, but it does not communicate the workload created by false alarms. Report the PR curve, the prevalence it was measured at, and the operating point you propose. [3 min]

Reference: https://doi.org/10.1371/journal.pone.0118432

Precision-recall exposes the alert trade-off

EVALUATING RARE EVENTS

Precision-recall exposes the alert trade-off



Logistic regression on a synthetic rare-event validation set (make_classification, seed 42). The dashed line is the observed positive prevalence; AP summarizes ranking across thresholds.

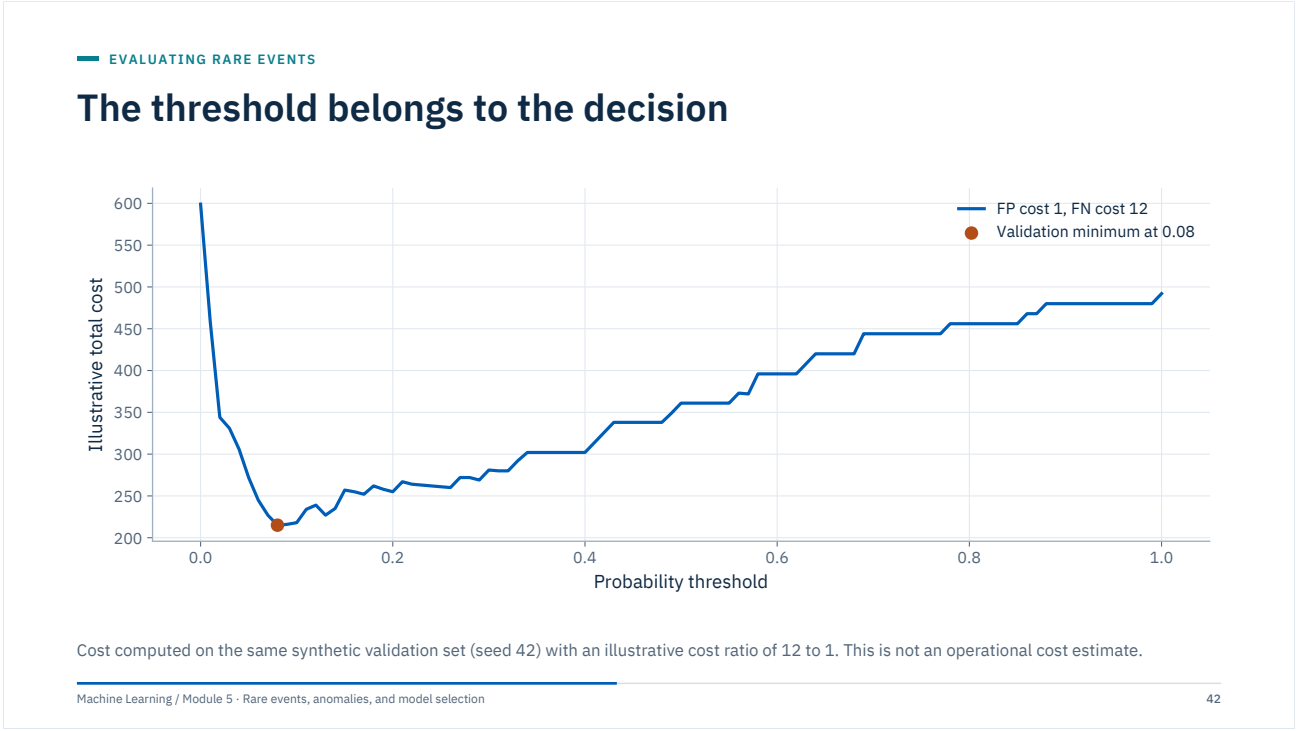
This curve comes from a logistic regression on a synthetic dataset with 6.4% positives. Reading from left to right: at low recall the highest-scored cases are almost all events; to catch more events we accept many more false alerts, and precision falls towards the prevalence line.

AP, or average precision, is not identical to every trapezoidal PR-AUC implementation. State the metric definition you use; scikit-learn's average_precision_score is a step-wise sum, not a trapezoid.

Keep the test prevalence representative, because precision changes with prevalence. ROC-AUC can be useful, but it does not directly communicate workload from false alarms.

Ask: where on this curve would you operate if every alert costs an engineer an hour and the team has four hours a day? That question leads to the next two slides. [2 min]

The threshold belongs to the decision



The curve adds up the cost of false positives (weight 1) and false negatives (weight 12) at each threshold on the validation set. The selected threshold minimizes the stated validation cost for this example, at about 0.08, far from the default 0.5.

The cost ratio is illustrative. In practice it comes from operations: what a missed trip costs compared with an unnecessary inspection, including production loss, repair and safety exposure, and those numbers are rarely known precisely. Show the decision is robust by recomputing with a range of plausible ratios.

Ask whether staff review capacity or a minimum recall constraint would change the decision. Often they do: a team that can handle ten alerts a week needs a threshold that yields about ten a week, whatever the cost ratio says.

Freeze the threshold before final test evaluation. Choosing it on the test set makes the reported precision and recall optimistic, the same selection problem we meet again in Part 5. [2 min]

With calibrated probabilities, costs set the threshold

EVALUATING RARE EVENTS

With calibrated probabilities, costs set the threshold

$$t^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

- Alert when the expected miss cost exceeds the false-alarm cost
- Costs 1 and 12 give t^* = 1/13 ≈ 0.077
- Close to the validation minimum of 0.08 on the previous slide
- Valid only if the probabilities are calibrated: see Part 4

◆ Review capacity can override costs: set the threshold to an alert rate the team can act on.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 43

Derivation: for a case with calibrated probability p of being an event, alerting costs (1 − p) × C_FP in expectation, while staying silent costs p × C_FN. Alert when p × C_FN > (1 − p) × C_FP, which rearranges to p > C_FP / (C_FP + C_FN). Here C_FP is the cost of a false alarm and C_FN the cost of a missed event.

With costs 1 and 12 the threshold is 1 / 13 = 0.077, which agrees with the empirical minimum of the cost curve at 0.08. The agreement is expected because that logistic regression was trained without reweighting and is roughly calibrated.

Two cautions. First, the formula assumes calibrated probabilities; after class weighting or undersampling the predicted probabilities are inflated (Part 4), and plugging them into this formula gives the wrong threshold. Second, costs are rarely known; if capacity is the binding constraint, set the threshold from the alert rate instead and report the implied cost ratio. [3 min]

Events differ from individual rows



Ten adjacent flagged samples may represent one alert, not ten independent successes. Operators see episodes, not rows. Event matching and warning windows must be defined before scoring, otherwise the evaluation can be tuned after the fact.

Event recall is the share of events preceded by at least one alert inside the warning window; useful warning time is how long before onset the first such alert came, and whether that is enough to act. False alerts should be counted as episodes per unit of exposure (asset-days, pump-months, kilometres of pipeline per week), which is what staffing depends on.

Specify any alert persistence, cooldown or deduplication rule, and apply exactly the same rule in validation and in deployment.

Do not claim early warning from features measured after the event starts. A model that uses a trip flag, a shutdown code or a post-event maintenance record as an input will look perfect and be useless. [3 min]

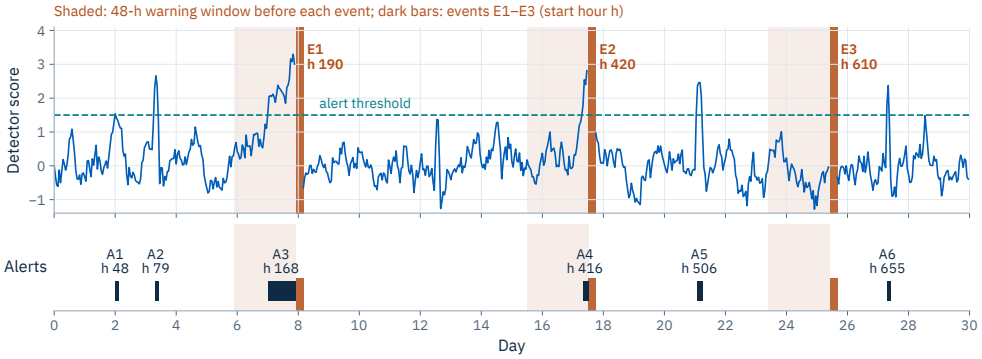
Score an alert log at the event level

PRACTICE

6 min · Pairs

Score an alert log at the event level

1. Which events were warned within 48 h, and with what lead time?
2. How many false alert episodes occurred per day?



Synthetic 30-day detector score (seed 22). Alert rows above the threshold are merged into episodes A1–A6 when gaps are 6 h or less; h = start hour.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 45

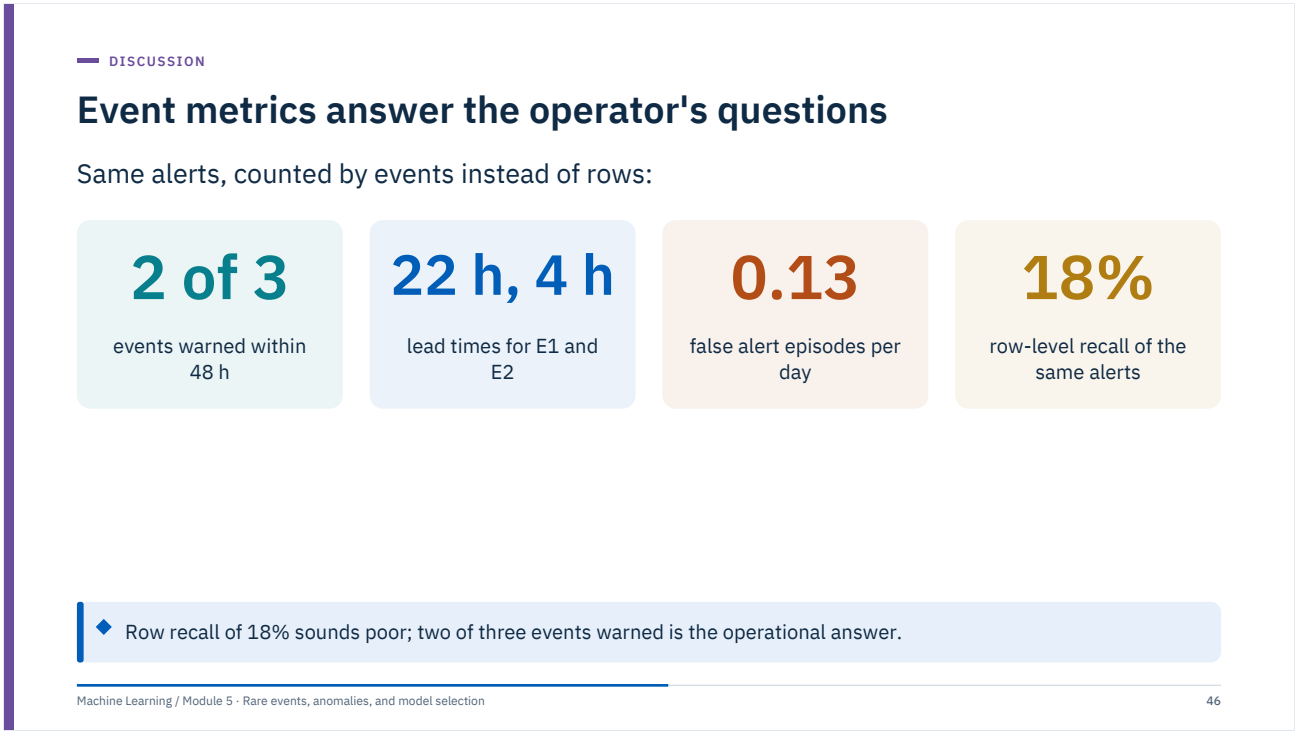
Give pairs four minutes. The labels give start hours so the arithmetic is exact.

Answer 1: E1 starts at hour 190; episode A3 starts at hour 168, inside the 48-hour window, so E1 is warned with a lead time of 190 − 168 = 22 hours. E2 starts at hour 420; A4 starts at hour 416, a lead of 4 hours. E3 at hour 610 has no alert in its window and is missed. Event recall is 2 of 3.

Answer 2: A1 (hour 48), A2 (79), A5 (506) and A6 (655) start outside every warning window, so there are 4 false alert episodes in 30 days: 0.13 per day, about one a week.

Discussion points: is 4 hours of warning useful? For a pump trip it may be too short to act, so a stricter definition of "useful warning" could make E2 a miss. Row-level metrics tell a different story: 36 alert rows, of which 26 lie inside warning windows (row precision 72%), but those 26 cover only 18% of the 144 window rows (row recall 18%). Common error: counting A3's 22 alert rows as 22 detections. [6 min]

Event metrics answer the operator's questions



Summarize the exercise with the numbers. Event recall is 2 of 3; the lead times are 22 and 4 hours; there are 4 false episodes in 30 days, 0.13 per day. The row-level view of the same alerts gives recall 18% (26 of 144 window rows) and precision 72% (26 of 36 alert rows).

Neither view is wrong, but they answer different questions. Row metrics answer "how much of the precursor period is flagged?", which matters little to an operator. Event metrics answer "how many trips did we see coming, how early, and at what cost in false alarms?", which is the decision question.

With three events, event recall is extremely uncertain: 2 of 3 is compatible with a true recall anywhere from about 9% to 99% (95% Clopper–Pearson interval, 0.094 to 0.992). We return to intervals in Part 4.

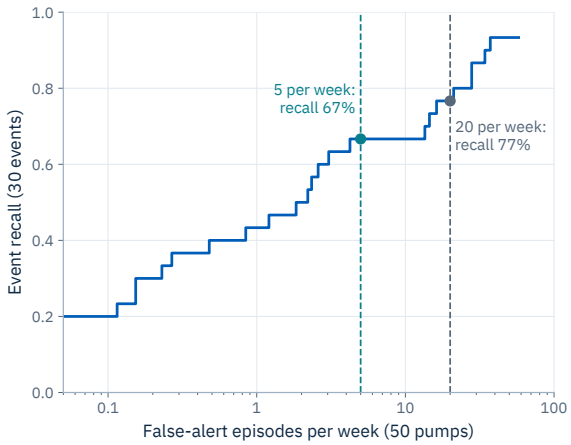
Transition: once events are counted properly, the threshold becomes a question of how many false episodes the team can absorb. [3 min]

Alert capacity chooses the operating point

EVALUATING RARE EVENTS

Alert capacity chooses the operating point

- Plot event recall against false alerts per week, not per row
- A team that reviews 5 false alerts a week gets 67% recall here
- Quadrupling the load to 20 a week buys only 77%
- Unreviewed alerts teach people to ignore the system



Synthetic fleet (seed 41): 50 pumps over 365 days, 30 events with precursors of random strength; alert episodes are runs of alert days outside warning windows.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 47

This curve is computed on a synthetic fleet of 50 pumps over a year with 30 distinct events. Each point is one threshold: its x-coordinate is the number of false alert episodes per week across the fleet, its y-coordinate the share of events warned within five days.

At 5 false alerts a week, 20 of 30 events are warned (67%); at 20 a week, 23 of 30 (77%). The curve flattens because some events have weak or no precursors in these features; no threshold will catch them, and lowering the threshold only adds false alarms.

Alert fatigue is the operational risk: when most alerts are false, people stop reading them, and the true ones are missed too. Choose the operating point with the people who will review the alerts, write down the expected alert rate, and monitor it after deployment. If the capacity-constrained recall is too low, the answer is better features or a better model, not a lower threshold.

Ask: how would you estimate this curve with only nine real trips? Answer: with wide uncertainty; resample by event or by pump, and report ranges. [3 min]

Without labels, judge a detector by stratified review

EVALUATING RARE EVENTS

Without labels, judge a detector by stratified review

Score band	Rows	Confirmed in a sample of 30	Estimated problems
Top: flagged	275	12	110
Middle	1,000	3	100
Bottom	4,218	1	141

Hypothetical review counts. Precision in the flagged band is 12 of 30 (40%), yet it holds only 110 of about 351 estimated problems (31%).

◆ Sample every band, unflagged rows included; review blind; weight each band by its size.

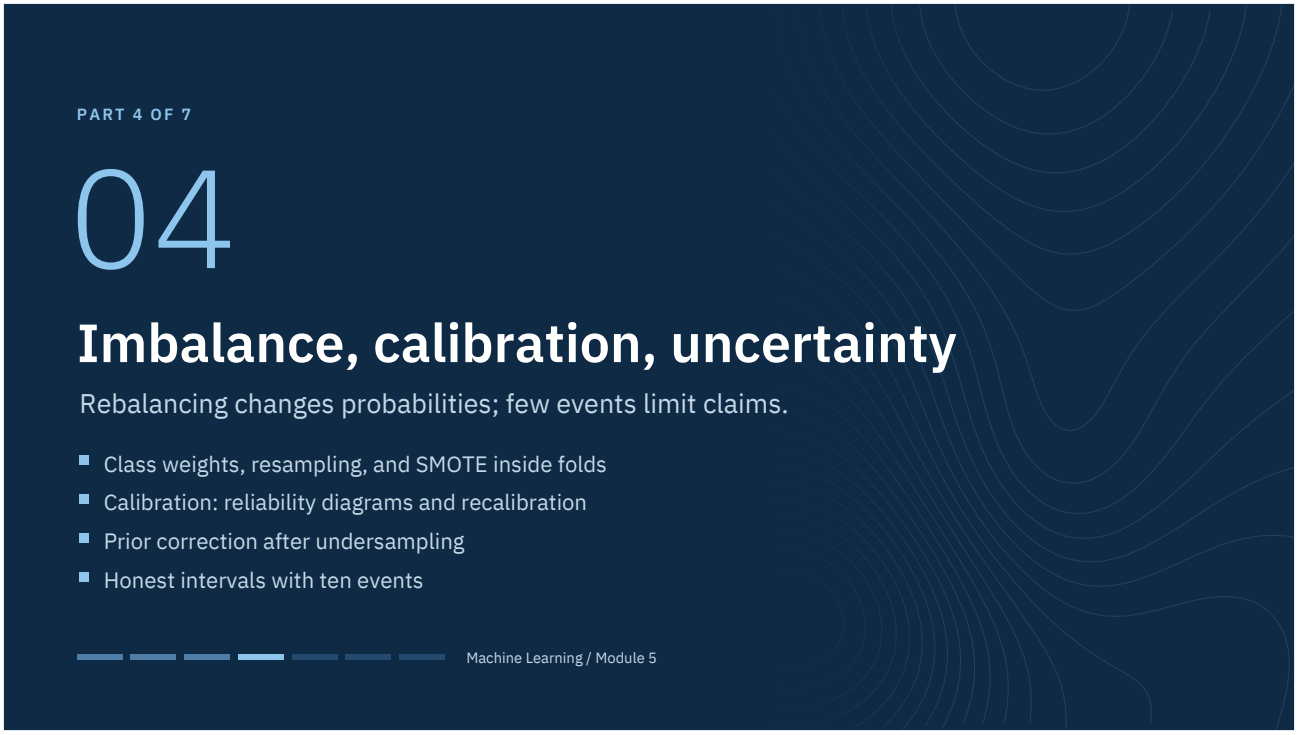
Without labels, do not invent precision or recall. Create labelled evidence by reviewing a stratified sample that includes unflagged rows; reviewing only alerts can estimate precision but never recall. Steps: rank with the frozen detector, split the ranking into bands, draw a fixed number of rows from each band, let experts label them without seeing the score, then weight each band's confirmed share by its size.

The table uses hypothetical review outcomes on the lab file. Weighting: 12/30 × 275 = 110, 3/30 × 1,000 = 100 and 1/30 × 4,218 = 141, about 351 problems in total. Precision in the flagged band is 40%, but that band holds only 110 / 351 = 31% of the estimated problems: most are spread thinly through the large unflagged bands.

The intervals are wide: 12 of 30 has a 95% Clopper–Pearson interval of 23% to 59%, and 1 of 30 an interval of 0.1% to 17%, which alone spans roughly 4 to 726 rows in the bottom band. Plan the sample size with that in mind.

Stability checks (the Jaccard overlaps of Lab 2) show whether a ranking is reproducible, not whether it is right. [3 min]

Imbalance, calibration, uncertainty



This part takes about 40 minutes after lunch. It covers supervised learning when positives are rare: what rebalancing does, what it costs, and how to keep probabilities meaningful.

Two messages to keep in mind. First, rebalancing changes the training problem, never the evaluation problem: validation and test sets keep the real prevalence. Second, rebalancing mostly changes where the threshold sits and how the probabilities are scaled; it rarely creates information that was not in the data.

The part ends with uncertainty. With ten events, even a perfect evaluation design gives wide intervals, and the report must say so. [1 min]

Each rebalancing strategy trades one risk for another

IMBALANCE, CALIBRATION, UNCERTAINTY

Each rebalancing strategy trades one risk for another

Strategy	Changes	Watch for
Class-weighted loss	Relative training penalty	Probability calibration changes
Random undersampling	Majority examples discarded	Loss of useful variation
Random oversampling	Minority examples repeated	Overfitting repeated cases
SMOTE	Interpolated minority features	Implausible combinations

Walk through the four strategies. Class weights multiply the loss of minority rows (class_weight="balanced" in scikit-learn weights each class inversely to its frequency). Undersampling discards majority rows, which is cheap but throws away variation. Oversampling repeats minority rows, which invites overfitting to those exact rows. SMOTE creates synthetic minority rows by interpolation (next slides).

Do not alter the final test prevalence to make scores look balanced. Threshold tuning can be valuable without changing training at all, as the next slide shows. Compare approaches under the same real evaluation distribution, with the same folds and the same metric, and include "no rebalancing, tuned threshold" as a baseline.

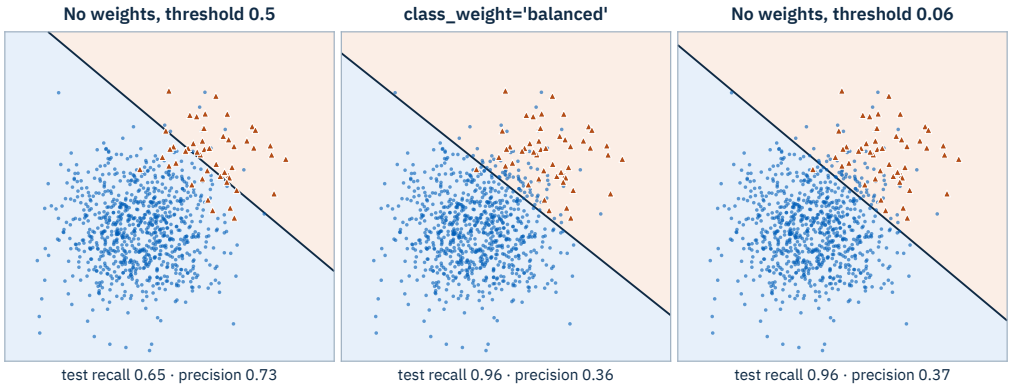
Ask: which of these would you try first for nine pump trips? A reasonable answer: none at first; fit a regularized model on the real data, tune the threshold on validation events, and only then test whether weighting helps. [3 min]

Class weights shift the decision boundary

IMBALANCE, CALIBRATION, UNCERTAINTY

Class weights shift the decision boundary

Balanced weights move the boundary much as a lower threshold would.



Synthetic 2-D data (seed 9), 5% events, 50/50 stratified split; logistic regression. Right: unweighted model with the threshold that matches the balanced model's training recall.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 51

Left: an unweighted logistic regression with the default threshold 0.5 catches 65% of test events with precision 0.73 (67 alerts). Middle: class_weight="balanced" moves the boundary towards the majority: recall 0.96, precision 0.36 (199 alerts). Right: the unweighted model with its threshold lowered to 0.06 gives almost the same boundary and almost the same result: recall 0.96, precision 0.37 (193 alerts).

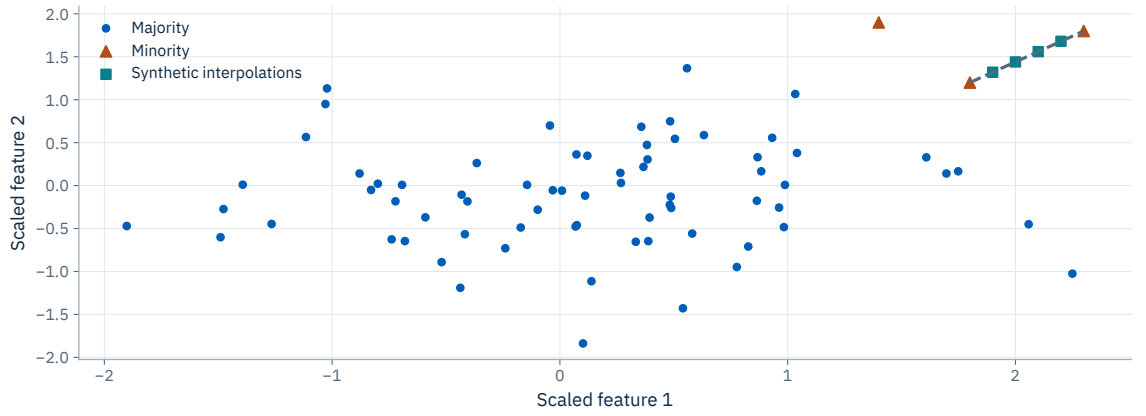
For logistic regression, weighting acts mostly like moving the threshold, and the threshold route keeps the probabilities calibrated. For other learners, such as trees and boosting, weights also change which splits are chosen, so they can change the ranking itself; test that rather than assume it.

Either way the decision about the operating point should be made explicitly, on validation data, with costs or capacity, not implicitly by the choice of weights. [3 min]

SMOTE interpolates in a chosen feature space

IMBALANCE, CALIBRATION, UNCERTAINTY

SMOTE interpolates in a chosen feature space



Synthetic points (seed 42): one SMOTE-style interpolation between two minority observations. The segment is mathematical interpolation, not evidence of physical feasibility.

SMOTE (Chawla et al., 2002) creates a synthetic minority row by picking a minority row, one of its nearest minority neighbours, and a random point on the segment between them. The formula is x_new = x_i + λ × (x_neighbor − x_i), with λ drawn uniformly between zero and one.

Standard SMOTE assumes numeric features. Interpolating across operating regimes, wells, or categorical encodings may generate invalid examples, for example a pump state halfway between running and shut down, or a depth sample halfway between two wells. Always ask whether the interpolated rows are physically possible.

SMOTE also inherits every problem of the feature space: unscaled features make the neighbour search meaningless, and noisy minority labels get multiplied. [2 min]

Resampling must happen inside training folds

IMBALANCE, CALIBRATION, UNCERTAINTY

Resampling must happen inside training folds

```python
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("scale", StandardScaler()),
    ("sample", SMOTE(random_state=42)),
    ("model", LogisticRegression(max_iter=1000))
])
```

Machine Learning / Module 5 · Rare events, anomalies, and model selection 53

Resampling before splitting leaks information: synthetic rows built from what will become test rows end up in training, and the test set no longer has the real prevalence. The fix is the same as for scaling in Module 1: put the sampler inside the pipeline so it runs only on each training fold.

This complete-data numeric example requires enough minority samples in every training fold for the SMOTE neighbour count (k_neighbors = 5 by default). Add imputation before scaling if needed. Use imbalanced-learn's Pipeline, because samplers act during fit only and are skipped during predict; the scikit-learn Pipeline does not support samplers. Keep group and time constraints in the cross-validation splitter.

Setup note: imbalanced-learn is a separate package (pip install imbalanced-learn). [2 min]

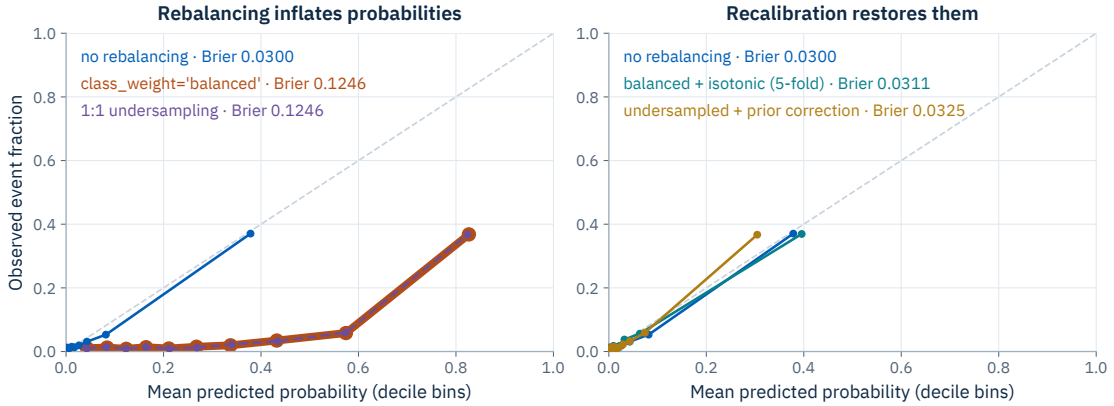
Reference: https://imbalanced-learn.org/stable/common_pitfalls.html

Reweighting and undersampling distort probabilities

IMBALANCE, CALIBRATION, UNCERTAINTY

Reweighting and undersampling distort probabilities

Mean probability 0.31 against an event rate of 0.055; the ranking barely moves.



Synthetic data (make_classification, seed 5, 40,000 rows, 5.5% events); logistic regression; reliability curves on the test half in decile bins.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 54

A reliability diagram groups test rows by predicted probability and plots the observed event fraction in each group. A calibrated model lies on the diagonal.

Left: the unweighted model is well calibrated (Brier score 0.0300). Class weighting and 1:1 undersampling both push the probabilities up: their mean predicted probability is 0.31 while the event rate is 0.055, and their Brier scores rise to 0.1246. The two curves almost coincide because both effectively shift the model's intercept by a similar amount.

The ranking is almost unchanged: average precision is 0.64 for the unweighted model and 0.61 for both rebalanced ones. Rebalancing changed the probabilities far more than the ability to rank.

Right: recalibration restores the probabilities. Isotonic regression fitted with 5-fold cross-validation gives Brier 0.0311; the prior correction for undersampling gives 0.0325. Niculescu-Mizil and Caruana (2005) compare calibration methods in detail.

Why it matters: cost-based thresholds, risk scores reported to engineers, and any averaging of probabilities all assume calibration. [3 min]

Recalibrate on rows the model did not train on

IMBALANCE, CALIBRATION, UNCERTAINTY

Recalibrate on rows the model did not train on

```
from sklearn.calibration import CalibratedClassifierCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import brier_score_loss
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

lr = LogisticRegression(class_weight="balanced",
                        max_iter=1000)
base = make_pipeline(StandardScaler(), lr)
cal = CalibratedClassifierCV(base, method="isotonic",
                             cv=5)
cal.fit(X_train, y_train)

p = cal.predict_proba(X_test)[:, 1]
print(brier_score_loss(y_test, p))
```

- Each fold fits the model, then a calibrator on held-out rows
- method="sigmoid" is Platt scaling: needs fewer events
- Isotonic is more flexible but needs more events
- Check a reliability plot and the Brier score on test rows

Machine Learning / Module 5 · Rare events, anomalies, and model selection 55

CalibratedClassifierCV with cv=5 fits the pipeline on four folds and a calibrator on the fifth, five times, and averages the calibrated predictions. The calibrator never sees rows its model was trained on, which is the essential point.

Platt scaling (method="sigmoid") fits a logistic curve to the scores: two parameters, stable with few events. Isotonic regression fits any monotone step function: more flexible, but it can overfit when there are only tens of events. With rare events, sigmoid is often the safer default; compare both on validation data.

If the model is already trained and you have a separate calibration set, scikit-learn 1.6 and later use FrozenEstimator (from sklearn.frozen) instead of the deprecated cv="prefit".

The Brier score is the mean squared difference between predicted probability and outcome; lower is better, and it rewards calibration as well as ranking. Keep the groups and time order in the cross-validation used for calibration too. [3 min]

Reference: https://scikit-learn.org/stable/modules/calibration.html

Undersampling needs a prior correction

IMBALANCE, CALIBRATION, UNCERTAINTY

Undersampling needs a prior correction

$$p = \frac{\beta p_s}{\beta p_s - p_s + 1}$$

- p_s: probability from the model trained on undersampled data
- β: fraction of majority rows kept (0.058 here)
- Restores the base rate: Brier 0.1246 falls to 0.0325
- Assumes the kept majority rows were sampled at random

◆ For class-weighted models there is no such shortcut: recalibrate on held-out rows.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 56

Derivation in one line: keeping each majority row with probability β multiplies the odds of the minority class in the training sample by 1/β. Undoing that gives odds = β × odds_s, which rearranges to the formula on the slide (Dal Pozzolo et al., 2015).

In the reliability example the 18,901 majority rows of the training half were undersampled to match its 1,099 events, so β = 1,099 / 18,901 = 0.058. Applying the correction brings the mean predicted probability back near the event rate (0.051 against 0.055) and the Brier score from 0.1246 to 0.0325.

The correction assumes the discarded rows were a random sample of the majority class. If undersampling was stratified by pump, period or regime, the correction is only approximate, and recalibration on held-out rows is safer.

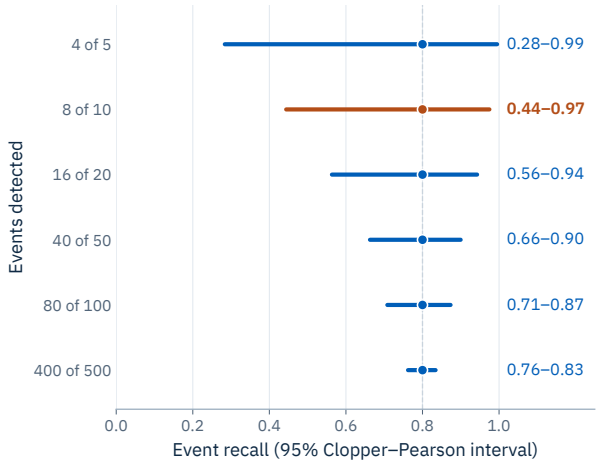
Ask: why not apply the same formula after class weighting? Because weighting changes the fitted coefficients as well as the intercept, so there is no exact closed form; use CalibratedClassifierCV. [3 min]

Few labelled events require honest uncertainty

IMBALANCE, CALIBRATION, UNCERTAINTY

Few labelled events require honest uncertainty

- Count independent events, not just labelled rows
- Review a sample of alerts and of non-alerts
- Resample whole events or assets, not rows
- State what the labels cannot estimate



Exact Clopper–Pearson 95% intervals for an observed recall of 80%.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 57

Reviewing only alerts creates verification bias and cannot estimate recall: you need to review a sample of non-alerted periods too. Bootstrap whole events or assets where appropriate, rather than treating correlated samples as independent. Do not fabricate confidence from thousands of rows representing only a few failures.

The figure shows how little ten events can tell you. An observed recall of 8 of 10 has a 95% Clopper–Pearson interval from 0.44 to 0.97. With 100 events the same observed rate narrows to 0.71 to 0.87, and with 500 events to 0.76 to 0.83. The Clopper–Pearson interval is exact but conservative; the Wilson interval is a common alternative.

A useful special case: if a detector warns of all n events, the one-sided 95% upper bound on the miss rate is 1 – 0.05^(1/n), roughly 3/n (the rule of three). For 10 of 10 that is 26%, so even a perfect record on ten events allows a miss rate of one in four. [3 min]

Eight of ten events detected: what can you claim?

PRACTICE

4 min Pairs

Eight of ten events detected: what can you claim?

A detector warned of 8 of the 10 pump trips in the validation year.

1. Is "recall = 80%" a fair summary for the report?
2. What is the 95% interval, and what would 80 of 100 give?
3. Name one way the ten trips might not be independent.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 58

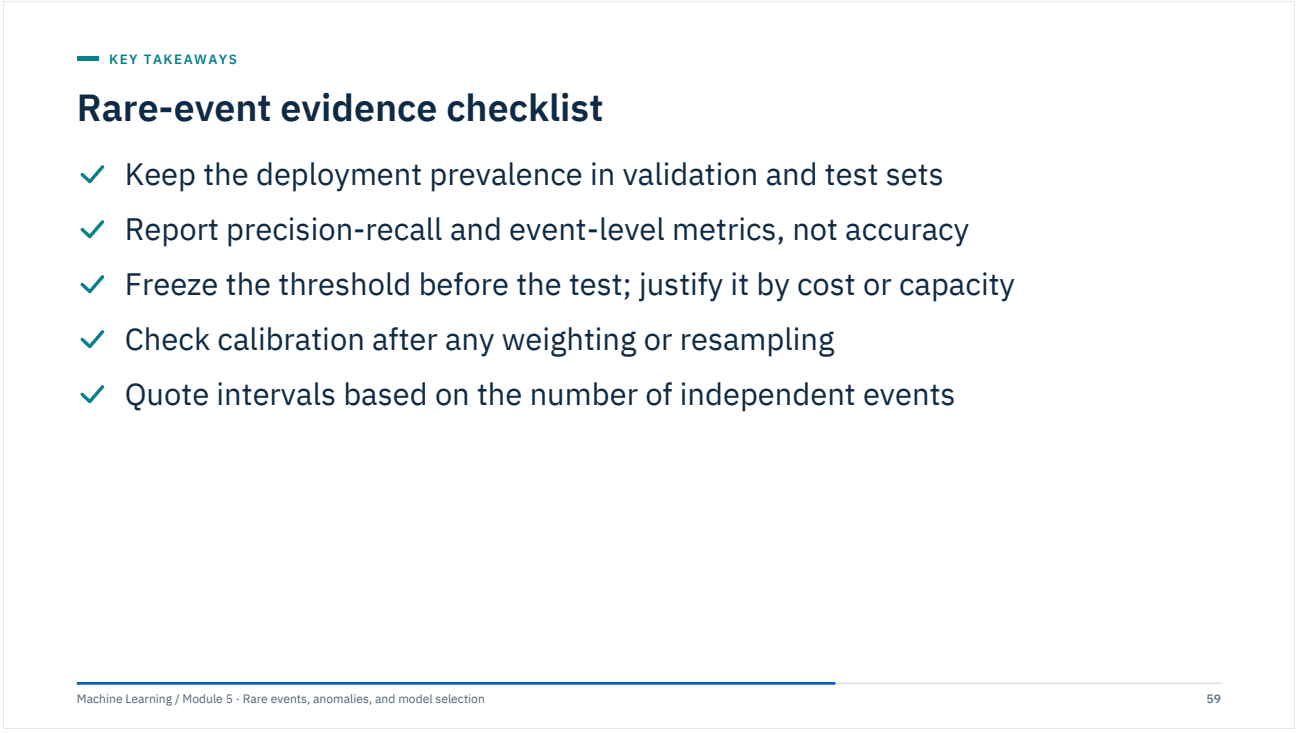
Answer 1: 80% is the point estimate, but reported alone it overstates what is known. A fair summary is "8 of 10 trips warned (95% interval 44% to 97%)", together with the warning window and the false-alert rate at the same threshold.

Answer 2: from the previous slide, 8 of 10 gives 0.44 to 0.97 (exactly 0.444 to 0.975); 80 of 100 would give 0.71 to 0.87 (0.708 to 0.873). Ten times more events narrows the interval roughly threefold, because the width shrinks with the square root of the number of events.

Answer 3: several trips on the same pump, trips caused by one bad batch of fluid or one power event, trips during the same workover campaign, or a detector tuned on similar trips last year. Dependence means the effective number of events is smaller than ten and the interval should be wider; resampling by pump rather than by trip is one remedy.

Common error: computing an interval from the number of labelled rows (for example 60 rows for 10 trips), which gives a falsely narrow interval. [4 min]

Rare-event evidence checklist



Read the checklist as the standard for Lab 2's alert policy and for any rare-event report.

Prevalence: rebalancing is allowed in training folds only. Metrics: precision-recall and event-level recall with lead time and false alerts per exposure; accuracy and ROC-AUC alone are not enough. Threshold: chosen on validation data from costs or capacity, frozen, then tested once. Calibration: checked with a reliability plot whenever probabilities are used as probabilities. Uncertainty: intervals from the number of independent events, not from rows.

Ask each table to pick the item most often missing in reports they have read. Common answers: intervals and the prevalence behind a precision figure.

Transition: Lab 2 finishes the notebook and asks you to write an alert policy that meets this checklist as far as unlabelled data allow. [2 min]

An alert policy turns scores into accountable actions



Brief the policy before the lab starts, so pairs can collect evidence for it while they work. This briefing is the first 5 of the 75 lab minutes.

The detector produces a score from trailing features, meaning features computed only from data available at the time of scoring. The policy then decides what becomes an alert. The threshold comes from costs or review capacity, as in Part 3. Persistence requires the score to stay above the threshold for several rows (for example 3 of 4 hours) so that single-row noise does not alert. Cooldown suppresses repeated alerts on the same asset for a period (for example 12 hours), so one episode produces one alert. The alert itself carries context: which features drove the score, recent trends, and similar past alerts.

The feedback loop matters most. Every reviewed alert produces a label (true event, known benign operation, sensor fault, unknown), and those labels are the evidence for the next threshold review.

For the well-log lab, translate "asset" to "well" and "hours" to depth intervals: persistence becomes a minimum interval thickness, and the reviewer is a petrophysicist. [5 min]

Lab 2: compare detectors and write an alert policy

LABORATORY

75 min

Lab 2: compare detectors and write an alert policy

- [ ] Part 4: LOF with n_neighbors from 5 to 100; compare flagged sets, not counts
- [ ] Part 5: One-Class SVM (nu = 0.05); plot its score track
- [ ] Part 6: overlap matrix, votes, and a rank-averaged review list
- [ ] Write a half-page alert policy with a threshold rationale

NOTEBOOK

`exercises/anomaly_detection/Anomaly_Detection_Student.ipynb`

EVIDENCE TO SUBMIT

- Overlap matrix of the five methods
- Jaccard overlap of LOF sets across n_neighbors
- Alert policy: score, threshold, rules, capacity, limits

Machine Learning / Module 5 · Rare events, anomalies, and model selection 61

Timing: 5 minutes of policy briefing (previous slide), 50 minutes of work, 10 minutes of debrief, and 10 minutes of slack. Suggested split: 15 minutes for Part 4, 10 for Part 5, 10 for Part 6 and 15 for the policy.

Part 4: the sweep counts flagged rows for n_neighbors = 5, 10, 20, 50 and 100, then computes the Jaccard overlap (size of the intersection divided by the size of the union) between every pair of flagged sets and shows it as a heatmap. Every count is 275, because contamination = 0.05 fixes it. Reference run (identical with scikit-learn 1.5, 1.7 and 1.9): n = 5 versus n = 20 gives 0.19, n = 20 versus n = 100 gives 0.26, and n = 5 versus n = 100 only 0.06.

Part 5: the SVM sits in a pipeline with its own scaler, so the scaler is always fitted on the rows the SVM is fitted on: all 5,493 here, or a random 10,000 when a larger table triggers the subsampling branch. OneClassSVM(nu=0.05, gamma="scale") flags 279 rows (5.1%): nu is a budget too, and the flagged share is close to it but not equal. The fit takes a few seconds.

Part 6: the overlap matrix (a seaborn heatmap), the vote counts, and a review list: the three learned detectors' scores averaged as percentile ranks, as on the rank-combination slide, keeping the top 50 rows. In the reference run those rows come from 15/9-F-1 B (30), 15/9-F-1 A (12) and 15/9-F-11 A (8), and 23 of them are flagged by all five methods. Many are adjacent samples of one interval (seven samples from 3,265.1 to 3,265.7 m in 15/9-F-1 B are all in the top ten), which is why the policy should count intervals, not rows. A closing note lists what the comparison cannot show: without labels there is no precision or recall.

Policy template (half a page): what is scored (rows, features, wells); the score and its direction; the threshold and why (capacity, cost or reviewed intervals); rules (persistence, cooldown, minimum interval thickness); who reviews, within what time; what evidence exists and what does not (no labels yet); when the policy is reviewed. Common errors: equating contamination with prevalence; using LOF predict without novelty=True; comparing raw scores across methods. [75 min]

Detectors agree on a core and differ at the margins

DISCUSSION

Detectors agree on a core and differ at the margins

- IF and One-Class SVM share 173 rows; LOF overlaps least
- LOF always flags 275 rows, but which rows changes with n_neighbors
- 24 rows are flagged by all five methods; 322 by two or more
- Agreement is not validation: no label confirms any flag yet

	|z| > 3	IQR	IF	LOF	OCSVM
|z| > 3	149	149	105	26	84
IQR	149	522	164	48	132
IF	105	164	275	68	173
LOF	26	48	68	275	74
OCSVM	84	132	173	74	279

diagonal: rows flagged by each method;
off-diagonal: rows flagged by both

Computed from well_logs_data.csv (course data), 5,493 complete rows, lab settings; scikit-learn 1.7.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 62

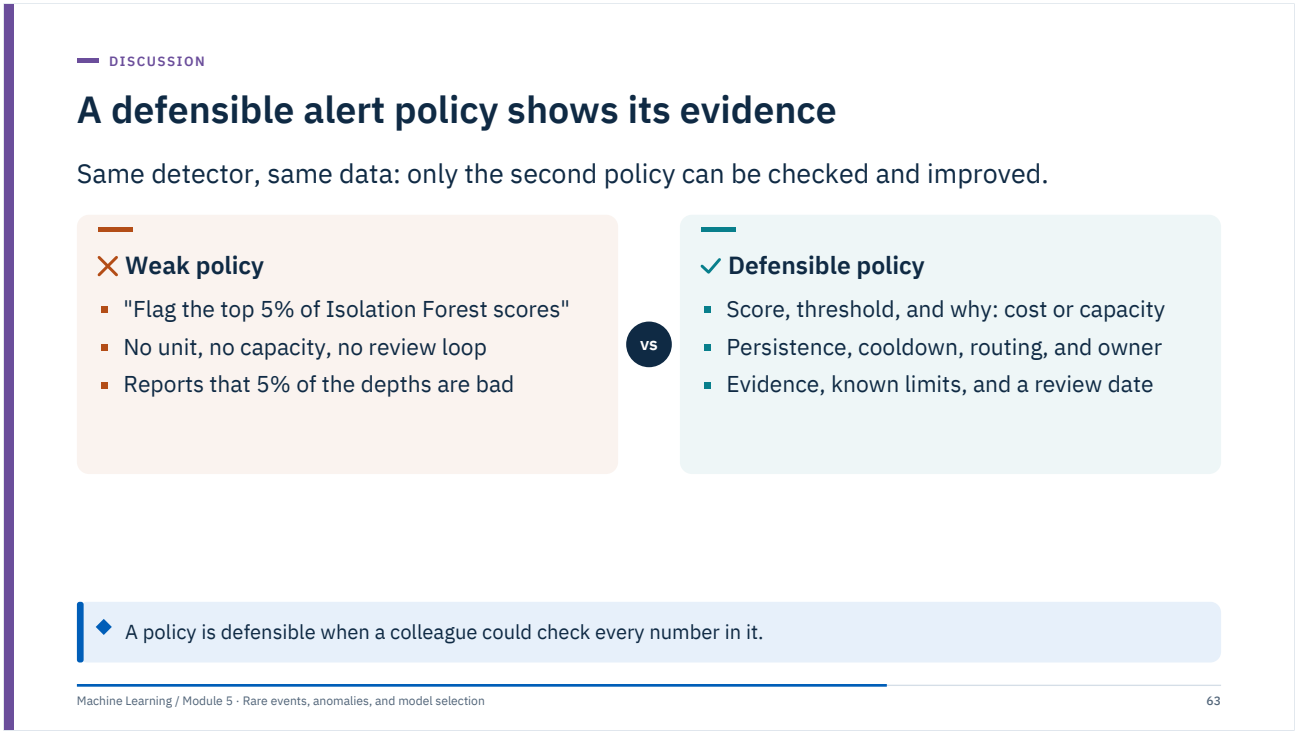
Read the matrix: the diagonal gives each method's count (149, 522, 275, 275 and 279); off-diagonal cells count rows flagged by both. Isolation Forest and One-Class SVM agree most (173 shared rows), since both describe the global shape of the data. LOF shares only 68 rows with Isolation Forest and 26 with the z-score rule: it finds rows that are unusual relative to their own neighbourhood, which is a different question.

Votes across the five methods: 4,599 rows get no vote, 572 one, 147 two, 90 three, 61 four and 24 all five; 322 rows have two or more votes. A consensus is a reasonable review order, not evidence of error: methods that share assumptions, like the IQR and z-score rules, agree for the same reasons and can be wrong together.

LOF sensitivity: the counts were constant at 275 for every n_neighbors, but the Jaccard overlap between n = 5 and n = 100 was 0.06: almost entirely different rows. The count was fixed by contamination; the neighbour count decided the meaning.

Next step: a petrophysicist reviews a stratified sample, as in Part 3, and those labels choose the policy. [5 min]

A defensible alert policy shows its evidence



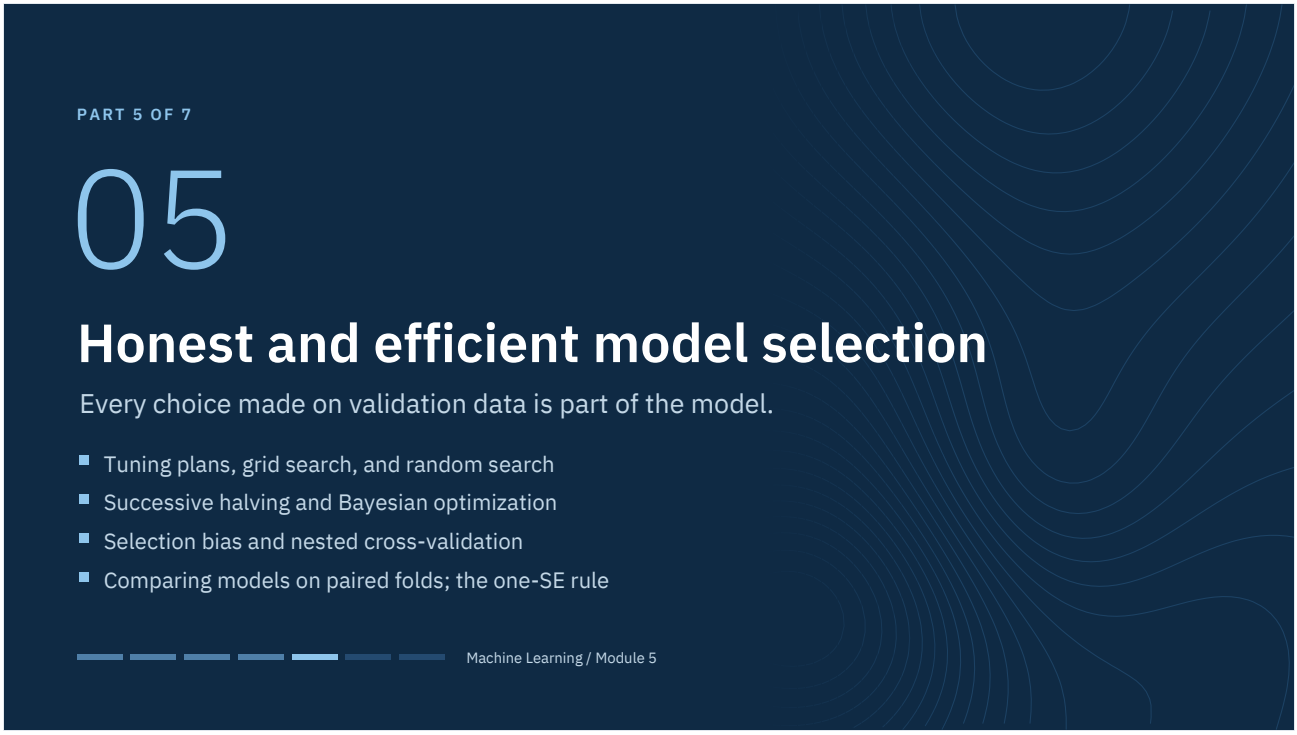
Ask two pairs to read their policies aloud and let the room place them on this spectrum. Then summarize with the two columns.

The weak policy is not wrong because of the number 5%; it is weak because nothing justifies it, nothing says who acts, and the conclusion confuses a setting with a finding.

A defensible policy for the well-log case could say: score = consensus of Isolation Forest and One-Class SVM, computed per well on standardized GR, RHOB, NPHI, DT and CALI; threshold = the top 50 intervals per well, because one petrophysicist can review about that many in a day; rule = flag only intervals at least 1 m thick; exclusions = shale-rich intervals are reviewed separately because the univariate rules flag them for geological reasons; evidence = none yet, the first review will produce labels; review date = after the first 200 reviewed intervals.

Assessment: this written policy is the main evidence for learning outcome 4. [5 min]

Honest and efficient model selection



This part takes about 40 minutes, including the tuning-log exercise. It applies to every supervised model in the course, and it is where rare-event projects most often fool themselves, because few events make validation scores noisy and noise rewards the lucky candidate.

Return to the distinction among model weights, hyperparameters, and the selected threshold. All choices informed by validation can overfit that validation evidence if repeatedly optimized. Feature selection, preprocessing options and even the choice of model family count as tuning.

The part has two halves: how to search efficiently (grid, random, halving, Bayesian), then how to report honestly (selection bias, nested cross-validation, statistical comparison, the one-standard-error rule). It ends with a paper exercise on reading a tuning log. [1 min]

A tuning plan defines the experiment

HONEST AND EFFICIENT MODEL SELECTION

A tuning plan defines the experiment

1. **Fix the question** — target, prediction-time features, and the split policy
2. **Choose the yardstick** — one primary metric plus practical constraints
3. **Bound the search** — candidate space, log scales, compute budget, stopping rule
4. **Keep a ledger** — record every trial, including failures and reruns
5. **Protect the test** — the final partition is used once, at the end

◆ The ledger shows how many candidates were tried, which is what the winner's score must be judged against.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 65

Run a small end-to-end baseline before searching: a pipeline with default settings, evaluated with the final split design. It tells you whether the problem is learnable at all and gives the tuning a reference point.

Choose one primary metric that reflects the decision (average precision or event recall at a capacity, for rare events), plus constraints such as run time or interpretability. Use a logarithmic scale for parameters spanning orders of magnitude, such as regularization strength or learning rate. Do not search parameters that are irrelevant for the chosen estimator.

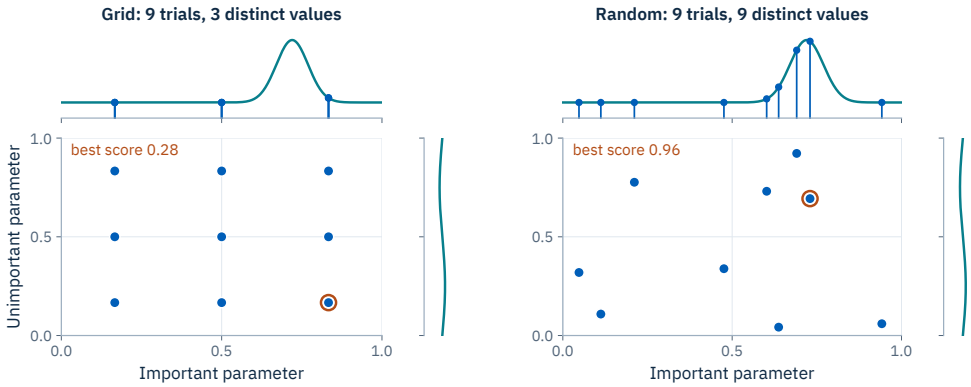
The ledger matters more than the search algorithm: record every configuration, its scores per fold, and the code version. Without it you cannot tell how many candidates were tried, which is exactly what you need to judge selection bias later in this part. [3 min]

Random search covers the parameters that matter

HONEST AND EFFICIENT MODEL SELECTION

Random search covers the parameters that matter

With 9 trials, a 3 × 3 grid tries 3 values of the important parameter; random search tries 9.



Synthetic objective with a narrow optimum on the important axis (teal curves). One random draw (seed 40); 9 random trials land within 0.05 of the optimum with probability 1 − 0.9 to the 9th power = 0.61.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 66

This is the argument of Bergstra and Bengio (2012). In many problems only one or two hyperparameters matter much, but we rarely know which in advance. A grid spends its budget repeating the same few values of each parameter: 9 trials give only 3 distinct values of the important one. Random search gives 9 distinct values of every parameter for the same cost.

In this example the score depends almost only on the important parameter, with a narrow optimum at 0.72. The grid's best trial scores 0.28; this particular random draw lands next to the optimum and scores 0.96. That was partly luck: the probability that at least one of 9 uniform trials falls within 0.05 of the optimum is 1 − 0.9^9 = 0.61, while the grid can never get there.

Practical rules: sample on a log scale for scale parameters, use more trials when more parameters are uncertain, and give random search the same validation design as any other method. [3 min]

Reference: https://www.jmlr.org/papers/v13/bergstra12a.html

Random search explores a bounded space

HONEST AND EFFICIENT MODEL SELECTION

Random search explores a bounded space

```
from scipy.stats import loguniform
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(
    model, {"model__C": loguniform(1e-3, 1e2)},
    n_iter=12, scoring="average_precision",
    cv=cv, random_state=42, n_jobs=-1
)
search.fit(X_train, y_train)
```

Pass groups to fit when cv requires asset or well groups.

Here model is the pipeline from the resampling slide in Part 4 (scaler, sampler, logistic regression), and cv is the chosen validation splitter, for example GroupKFold by well or a time-series split. The parameter name model__C addresses the C parameter of the step named "model" in the pipeline.

loguniform samples C evenly on a log scale between 0.001 and 100, so each order of magnitude gets a similar share of the 12 trials. The scoring is average precision, the rare-event metric from Part 3. random_state makes the sampled candidates reproducible.

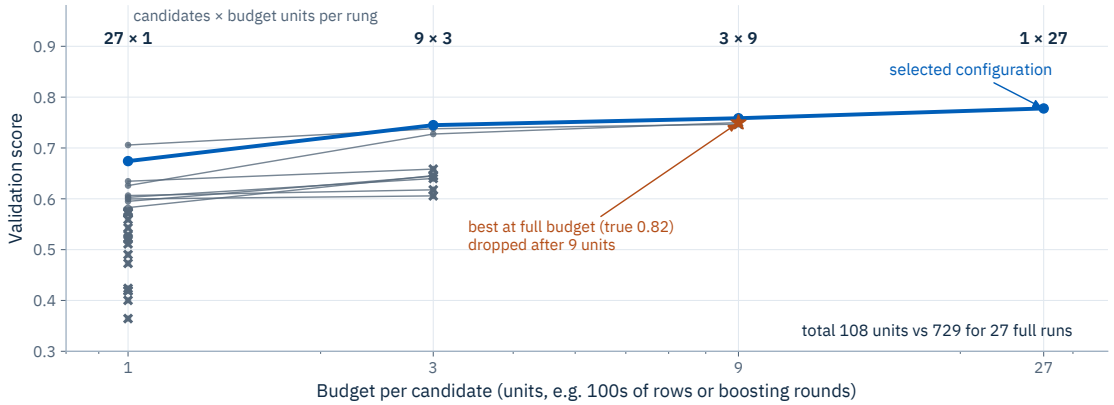
The code shows distribution sampling, not an endorsement of SMOTE for every dataset. Evaluate the returned best estimator on separate data, and select a threshold on separate validation evidence if necessary. If cv needs groups, call search.fit(X_train, y_train, groups=wells_train). [2 min]

Successive halving spends budget on survivors

HONEST AND EFFICIENT MODEL SELECTION

Successive halving spends budget on survivors

Start many candidates cheaply, keep the best third, and triple the budget of the survivors.



Simulated learning curves for 27 candidates (seed 8); the top third is kept at each rung (factor 3).

Successive halving (Jamieson and Talwalkar, 2016) evaluates many candidates with a small budget, such as a fraction of the training rows or a few boosting rounds, keeps the best third, triples their budget, and repeats. Here 27 candidates go through rungs of 1, 3, 9 and 27 budget units: 27 × 1 + 9 × 3 + 3 × 9 + 1 × 27 = 108 units, against 27 × 27 = 729 for giving every candidate the full budget.

The risk is visible in the figure: the candidate that would be best at full budget (true score 0.82) learns slowly, and it was dropped at the 9-unit rung in favour of a faster starter. Halving assumes that early rankings predict final rankings. Hyperband (Li et al., 2018) hedges by running several halving schedules with different starting budgets.

In scikit-learn: from sklearn.experimental import enable_halving_search_cv, then HalvingRandomSearchCV(model, params, factor=3, resource="n_samples"). The resource can also be a parameter such as n_estimators. It is still marked experimental. [3 min]

Reference: https://scikit-learn.org/stable/modules/grid_search.html

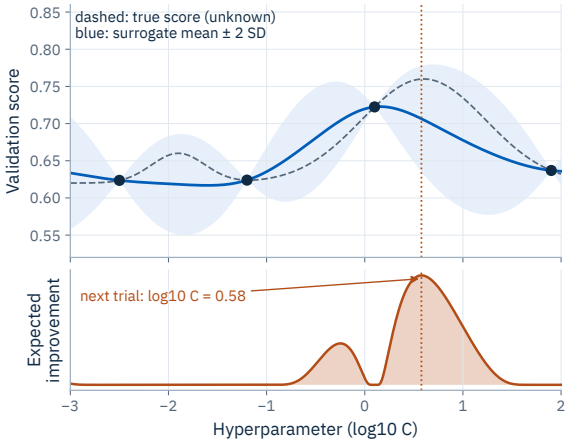
Bayesian optimization uses earlier trial results

HONEST AND EFFICIENT MODEL SELECTION

Bayesian optimization uses earlier trial results

$$\mathrm{EI}(x) = E[\max(f(x) - f^{+}, 0)]$$

- A surrogate estimates performance across the search space
- The acquisition rule balances promising and uncertain settings
- Each completed trial updates the next choice
- Keep the same validation design; record the whole budget



Synthetic 1-D objective; scikit-learn GaussianProcessRegressor (Matern kernel, fixed hyperparameters) after four trials.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 69

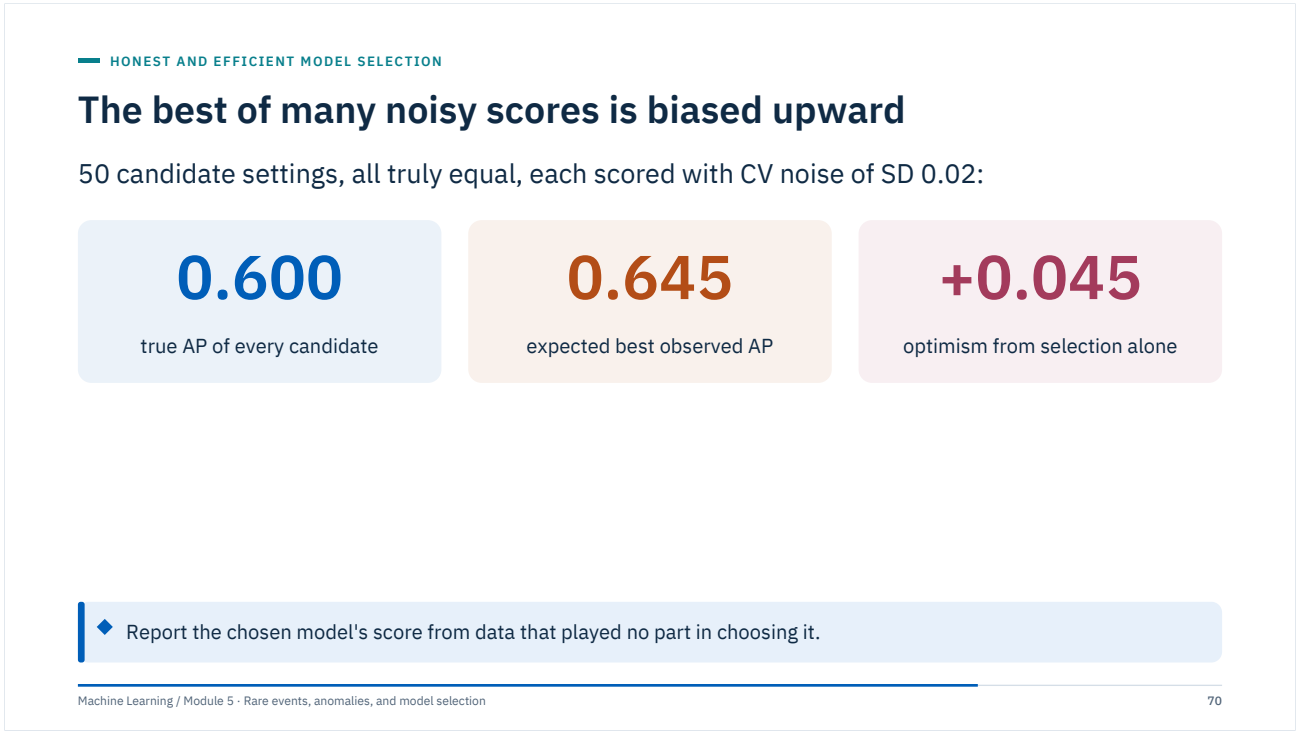
Define: f(x) is the validation score at setting x, f^{+} the best score observed so far, and EI the expected improvement over f^{+} under the surrogate model.

The surrogate here is a Gaussian process fitted to four trials. Its mean (blue) passes through the observed scores, and its uncertainty band widens between trials. The dashed curve is the true score, unknown in practice. Expected improvement is high where the mean is high, the uncertainty is large, or both; for a Gaussian process it has the closed form $(\mu - f^{+} - \xi)\Phi(Z) + \sigma\varphi(Z)$, with $Z = (\mu - f^{+} - \xi)/\sigma$, where μ and σ are the surrogate's mean and standard deviation at x, Φ and φ the standard normal distribution and density functions, and ξ a small margin that favours exploration (0.01 in the figure; with ξ = 0 this is exactly the expectation on the slide). The next trial goes to log10 C = 0.58, close to the true optimum at 0.6 (Jones et al., 1998; Snoek et al., 2012).

Bayesian optimization does not repair leakage or guarantee a global optimum. Noisy fold results and a large search can still create optimistic selection. It pays off when each trial is expensive and the number of hyperparameters is modest. Libraries such as scikit-optimize and Optuna implement related methods. [3 min]

Reference: https://scikit-learn.org/stable/modules/gaussian_process.html

The best of many noisy scores is biased upward



The numbers come from order statistics, not from a simulation of a particular dataset. If 50 candidates all have a true average precision of 0.60 and each cross-validated estimate has noise with standard deviation 0.02, the expected maximum of 50 standard normal values is 2.249, so the best observed score is expected to be 0.60 + 2.249 × 0.02 = 0.645. The winner looks 0.045 better than it is, purely because we picked the largest of 50 noisy numbers. With 10 candidates the expected maximum is 1.539 standard deviations; with 100 it is 2.508.

With few events, CV noise is large, so this optimism can exceed the real differences between candidates. Cawley and Talbot (2010) and Varma and Simon (2006) document how large this bias can be.

Two remedies follow: estimate the performance of the whole selection procedure with nested cross-validation (next slide), or keep a test set untouched until the choice is final. [2 min]

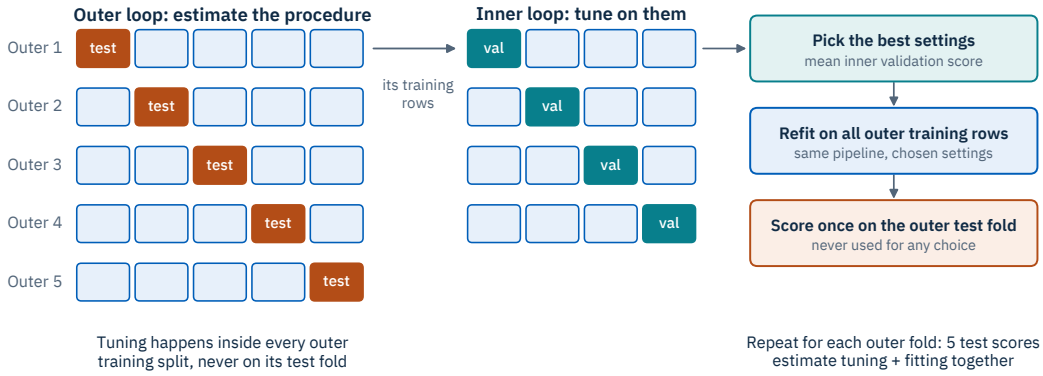
Reference: https://www.jmlr.org/papers/v11/cawley10a.html

Nested cross-validation scores the selection procedure

HONEST AND EFFICIENT MODEL SELECTION

Nested cross-validation scores the selection procedure

The outer score estimates how well tuning plus fitting generalizes, not one fixed model.



Schematic: five outer folds; inside each outer training split, four inner folds choose the settings.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 71

Nested cross-validation has two loops. The inner loop runs the whole tuning procedure (for example the random search from two slides ago) on the outer training rows only and picks the settings. The chosen configuration is refit on all outer training rows and scored once on the outer test fold. Repeating this for each outer fold gives five test scores that estimate the performance of the procedure "tune, then fit".

In code: cross_val_score(search, X, y, cv=outer_cv), where search is a RandomizedSearchCV with its own inner cv. Each outer fold may choose different settings; that variation is information about how stable the tuning is.

Nested cross-validation is an option when data is limited and an unbiased estimate of the selection procedure is needed. It costs outer × inner × candidates fits. Groups and time order must be respected in both loops; with scikit-learn's metadata routing, groups can be passed through to the inner search.

After nested CV, the model you deploy is the result of running the tuning once more on all the data; the nested score is its performance estimate. [3 min]

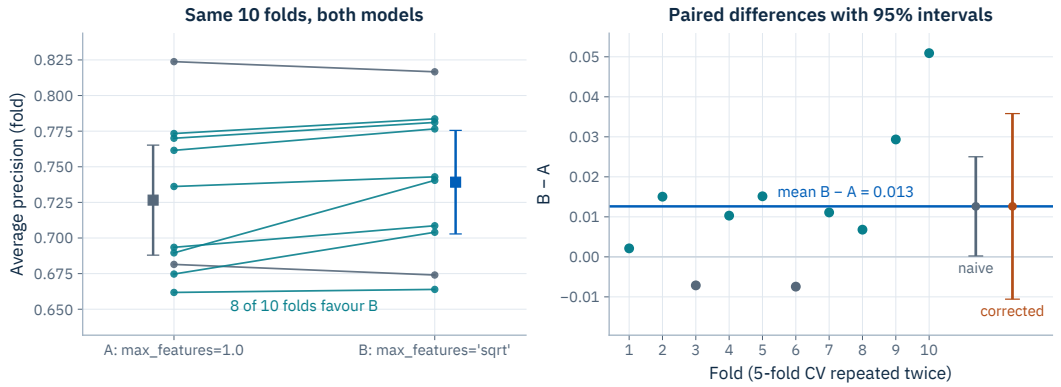
Reference: https://scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html

Compare models on paired folds, with a corrected test

HONEST AND EFFICIENT MODEL SELECTION

Compare models on paired folds, with a corrected test

The means' error bars overlap, yet B beats A in 8 of 10 paired folds.



Synthetic data (seed 4, 1,200 rows); RepeatedStratifiedKFold 5 × 2; two random forests differing only in max_features. Corrected interval: Nadeau–Bengio variance, test/train ratio 1/4.

Left: average precision for models A and B on the same ten folds. Fold difficulty varies a lot (from 0.66 to 0.82), so the 95% intervals of the two means overlap heavily: A 0.727 ± 0.039, B 0.739 ± 0.036. Comparing those intervals says "no difference" because it mixes fold difficulty into the comparison.

Right: the paired differences B − A remove fold difficulty. Their mean is 0.013 with an SD of 0.017, positive in 8 of 10 folds. A naive paired t-test gives a 95% interval of 0.000 to 0.025 (p = 0.047). But CV folds share training rows, so their scores are correlated and the naive test is overconfident. The corrected resampled t-test of Nadeau and Bengio (2003) inflates the variance by (1/k + n_test/n_train) instead of 1/k; here the interval becomes −0.011 to 0.036 (p = 0.25).

Conclusion: B is probably slightly better, but the evidence is weak. Choose on cost, simplicity or robustness, and say so. scikit-learn's example "Statistical comparison of models using grid search" shows the corrected test. [3 min]

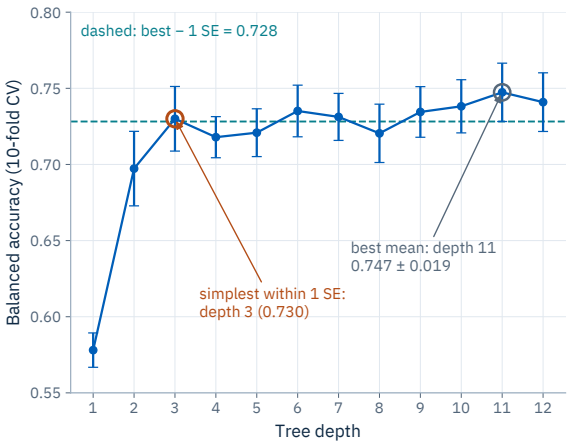
Reference: https://scikit-learn.org/stable/auto_examples/model_selection/plot_grid_search_stats.html

Prefer the simplest model within one standard error

HONEST AND EFFICIENT MODEL SELECTION

Prefer the simplest model within one standard error

- Find the best mean score and its standard error
- Choose the simplest model within one SE of that best
- Here depth 3 (0.730) instead of depth 11 (0.747)
- A heuristic: fold scores are correlated, so the SE is approximate



Synthetic data (seed 20, 800 rows); DecisionTreeClassifier; 10-fold stratified CV; bars show ±1 standard error.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 73

The one-standard-error rule, described by Breiman et al. for CART and in The Elements of Statistical Learning (section 7.10), picks the simplest model whose mean CV score is within one standard error of the best mean.

Here the best mean balanced accuracy is 0.747 ± 0.019 at depth 11, so the line is 0.728. Depth 3 scores 0.730, above the line, and is the simplest model that qualifies. The curve is a plateau from depth 3 to 12 with noise on top; the peak at depth 11 is mostly selection luck, the effect from two slides ago.

Why prefer depth 3: a shallower tree is easier to explain, cheaper to maintain, less sensitive to drift, and not measurably worse. The standard error is computed as the SD of the fold scores divided by √10, which ignores the correlation between folds, so treat the rule as a sensible default rather than a test.

Ask: what counts as "simplest" for a random forest or a pipeline? Examples: fewer features, shallower trees, stronger regularization, a model family the operators already trust. [3 min]

Read a tuning log and choose a model

PRACTICE

8 min Pairs

Read a tuning log and choose a model

Fold	Depth 3	Depth 6	Depth 12
1	0.61	0.66	0.67
2	0.58	0.62	0.61
3	0.64	0.69	0.70
4	0.55	0.60	0.62
5	0.62	0.68	0.70

1. Which depth has the best mean average precision?
2. Apply the one-SE rule (the SE of depth 12 is 0.019).
3. Is depth 12 clearly better than depth 6 on paired folds?

Machine Learning / Module 5 · Rare events, anomalies, and model selection 74

Paper exercise; allow calculators. Give six minutes, then work through it.

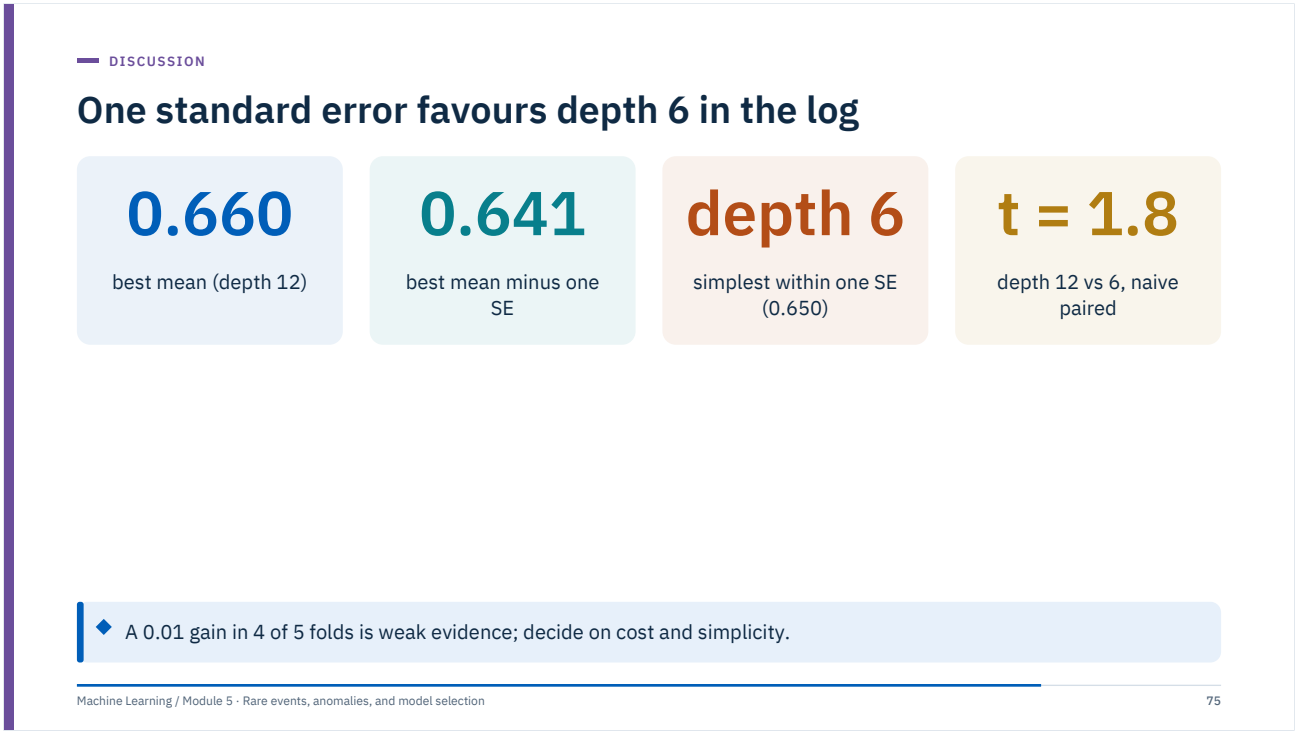
Answer 1: means are 3.00 / 5 = 0.600 for depth 3, 3.25 / 5 = 0.650 for depth 6, and 3.30 / 5 = 0.660 for depth 12. Depth 12 has the best mean.

Answer 2: the SD of the depth-12 scores is 0.0430 (deviations 0.01, −0.05, 0.04, −0.04, 0.04; squares sum to 0.0074; divided by 4 gives 0.00185), so SE = 0.0430 / √5 = 0.019. The line is 0.660 − 0.019 = 0.641. Depth 6 (0.650) is above it and depth 3 (0.600) below, so the rule picks depth 6.

Answer 3: paired differences depth 12 − depth 6 are 0.01, −0.01, 0.01, 0.02, 0.02: mean 0.010, SD 0.0122, SE 0.0055, t = 1.83 with 4 degrees of freedom (two-sided p about 0.14, before any correction for overlapping folds). Not clearly better. By contrast depth 6 − depth 3 gives 0.05, 0.04, 0.05, 0.05, 0.06: mean 0.05 with SD 0.007, a consistent gain.

Common errors: comparing depth 12 with the best mean minus its own SD instead of SE; ignoring the pairing; declaring depth 12 the winner because its mean is highest. [8 min]

One standard error favours depth 6 in the log

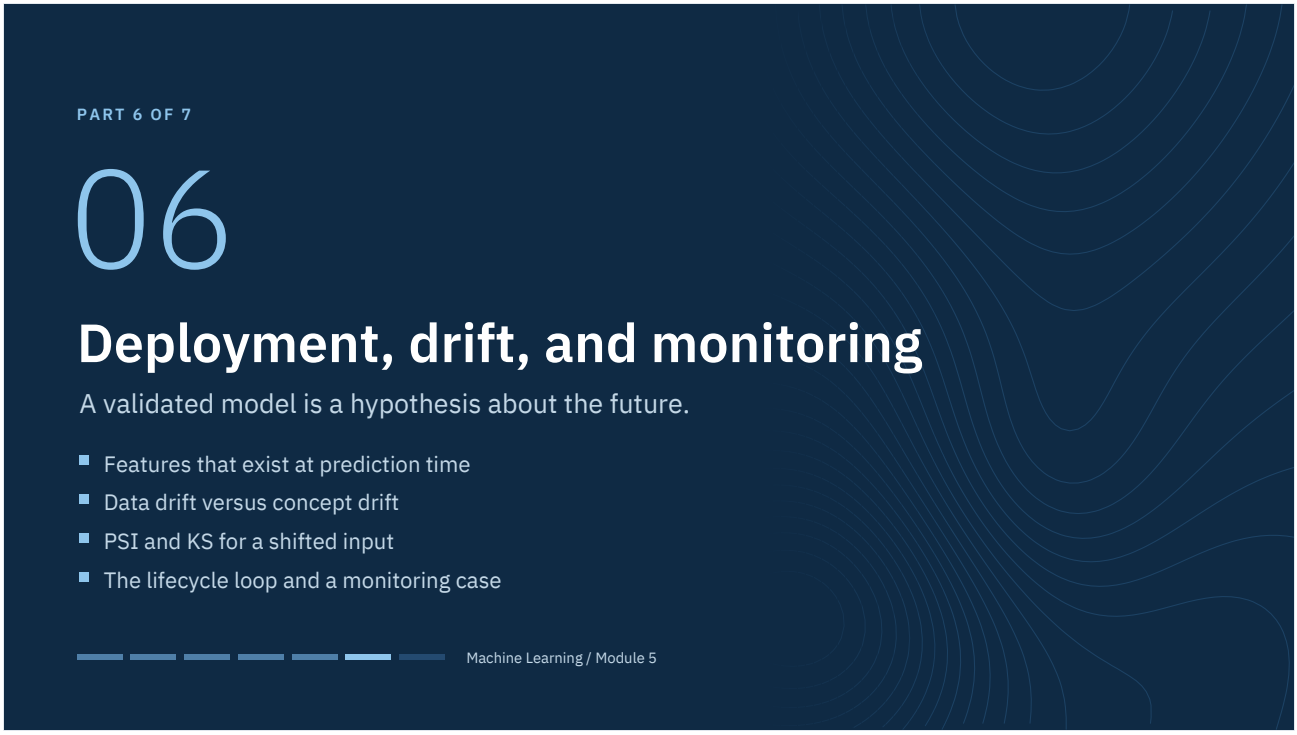


Summarize the exercise. Depth 12 has the best mean, 0.660, but only by 0.010 over depth 6, and the gain appears in 4 of 5 folds. The one-SE line at 0.641 admits depth 6 (0.650) but not depth 3 (0.600), so the rule chooses depth 6.

The paired analysis supports the same decision: the naive paired t-statistic is 1.83 with 4 degrees of freedom, not convincing even before correcting for overlapping training sets, which would weaken it further. The gain from depth 3 to depth 6, between 0.04 and 0.06 in every fold, is the real improvement in this log.

Ask: what else would you want in the log? Good answers: the number of candidates tried in total (for selection bias), the event counts per fold (with few events, AP is noisy), and per-well results. [3 min]

Deployment, drift, and monitoring



This part takes about 20 minutes. Validation tells us how a model performed on the past under a stated design; deployment tests whether that still holds. Two things can break it: features that were not really available at prediction time, and a world that changes after training.

We start with a timing exercise, because the most common deployment failure in forecasting projects is a feature that quietly used the future. Then we separate two kinds of change, show two simple drift statistics, and close the loop with monitoring. [1 min]

Feature timing determines whether a forecast is real

PRACTICE

4 min Pairs

Feature timing determines whether a forecast is real

A 30-minute warning model uses a centred 30-minute rolling mean, which includes the next 15 minutes.

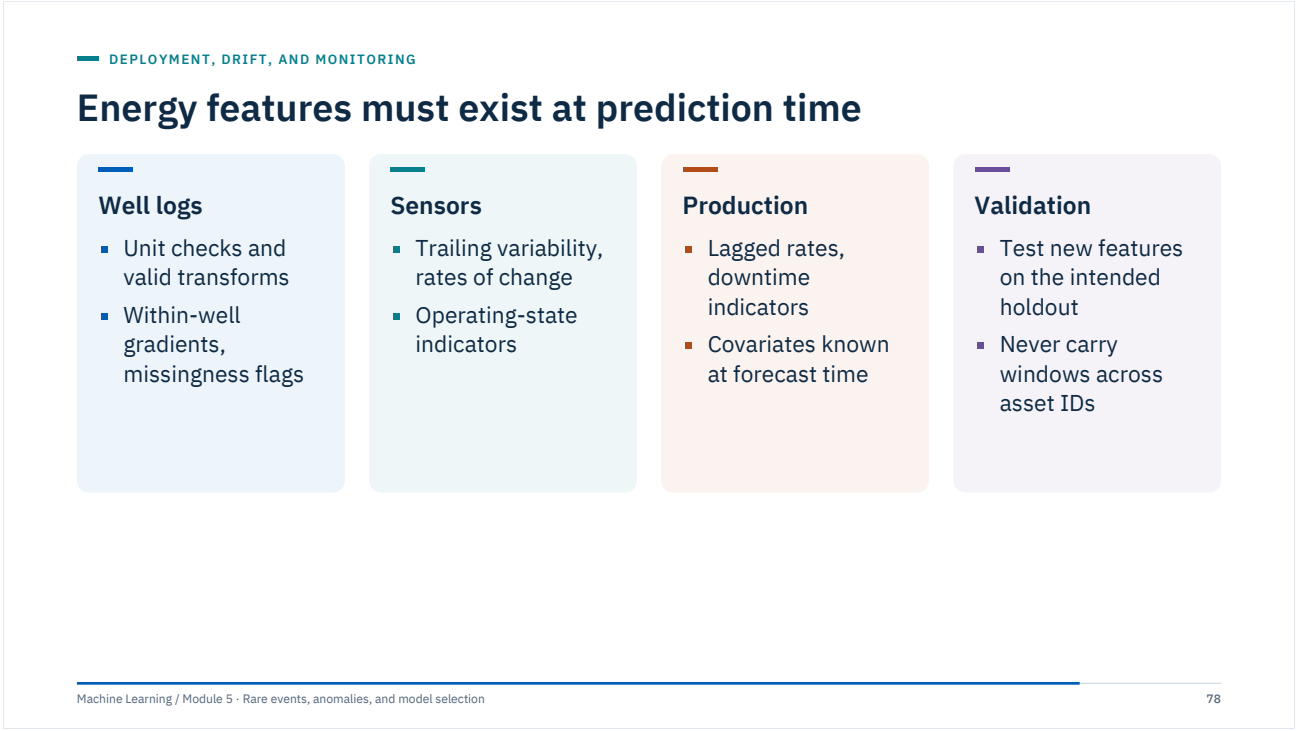
1. Is that feature available at the forecast origin?
2. How would you repair the feature and the validation design?

Answer 1: no. A centred window at time t uses values up to t + 15 minutes, which are unavailable at the forecast origin unless future observations are known independently (a planned setpoint, for example). The model looks good in validation because it sees part of the future it claims to predict.

Answer 2: use a trailing feature computed only from data up to t, such as the rolling mean of the previous 15 minutes, and shift the target to the intended horizon (the event state at t + 30 minutes). Split chronologically and exclude training targets that overlap the validation horizon. A gap between training and validation periods is useful when label windows overlap; choose it from the data construction, for example at least the horizon plus the longest feature window.

Common error: pandas rolling(..., center=True) or a filter applied to the whole series before splitting, which leaks in the same way. [4 min]

Energy features must exist at prediction time



Use this checklist when building features for any energy model in the course.

Well logs: check units (API, g/cc, microseconds per foot) before combining wells; log-transform resistivity with a defined treatment of zero and negative values; compute gradients within a well, never across well boundaries; keep missingness flags, since a missing curve often signals borehole conditions.

Sensors: trailing statistics (rolling mean, SD, slope) and operating-state indicators (running, starting, tripped) usually carry more information than raw values. Production: lagged rates and downtime indicators, plus only those covariates known at forecast time.

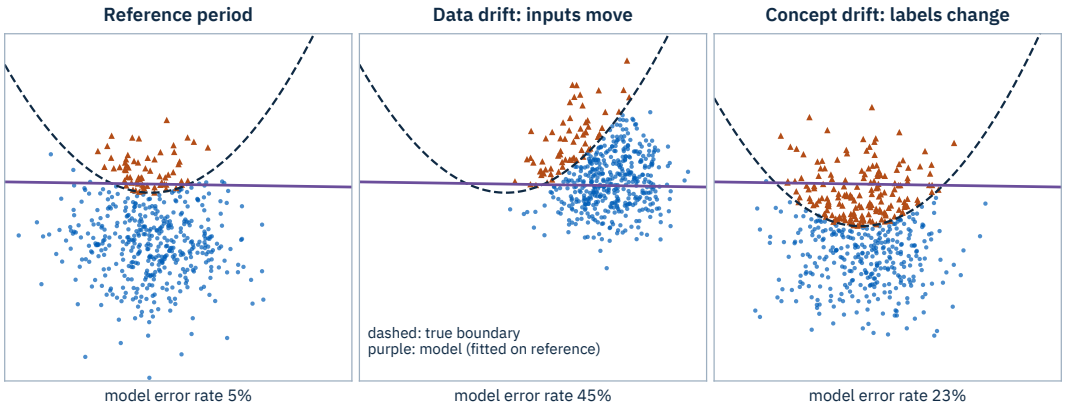
Physical features can help but do not invariably outperform raw inputs; validate added features on the intended well or time holdout. Protect train-validation boundaries when computing statistics, and avoid carrying rolling windows across asset IDs. [2 min]

Data drift and concept drift need different responses

DEPLOYMENT, DRIFT, AND MONITORING

Data drift and concept drift need different responses

Data drift: P(x) moves. Concept drift: P(y | x) changes. Either breaks a validated model.



Synthetic data (seed 14): events when x2 > 0.25 x1² + c; logistic regression fitted on the reference period only. Error rates on 500 rows per panel.

Left: in the reference period the model's straight boundary (purple) approximates the true curved boundary (dashed) well where the data are, with a 5% error rate.

Middle, data drift (covariate shift): the inputs move to a new region, for example a new operating envelope after a pump upgrade. The true relationship is unchanged, but the model now extrapolates, and its straight line no longer matches the curved truth there: the error rate jumps to 45%. Data drift is detectable from the inputs alone, without labels, which makes it the first thing to monitor.

Right, concept drift: the inputs look the same, but the relationship changes, for example equipment ageing makes events happen at lower stress. The model is now wrong where it used to be right (23% error). Concept drift is invisible in the inputs; detecting it needs outcomes, which arrive late in rare-event problems.

Responses differ: data drift calls for checking coverage and retraining with data from the new region; concept drift calls for new labels, then retraining or recalibration (Gama et al., 2014). [3 min]

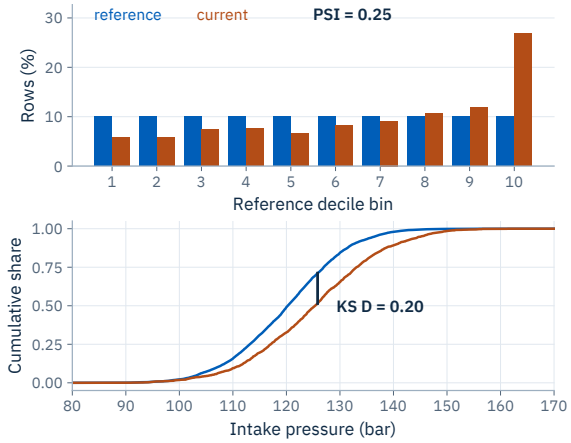
PSI and KS quantify a shifted input

DEPLOYMENT, DRIFT, AND MONITORING

PSI and KS quantify a shifted input

$$\mathrm{PSI} = \sum_i (a_i - e_i) \ln \frac{a_i}{e_i}$$

- e_i and a_i: reference and current shares in bin i
- Here PSI = 0.25 and KS D = 0.20 for a 5-bar shift
- With 1,500 rows the KS p-value is below 10^{-39}
- Set alert levels from PSI between past stable periods



Synthetic intake pressure (seed 15): reference N(120, 10), 5,000 rows; current N(125, 12), 1,500 rows.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 80

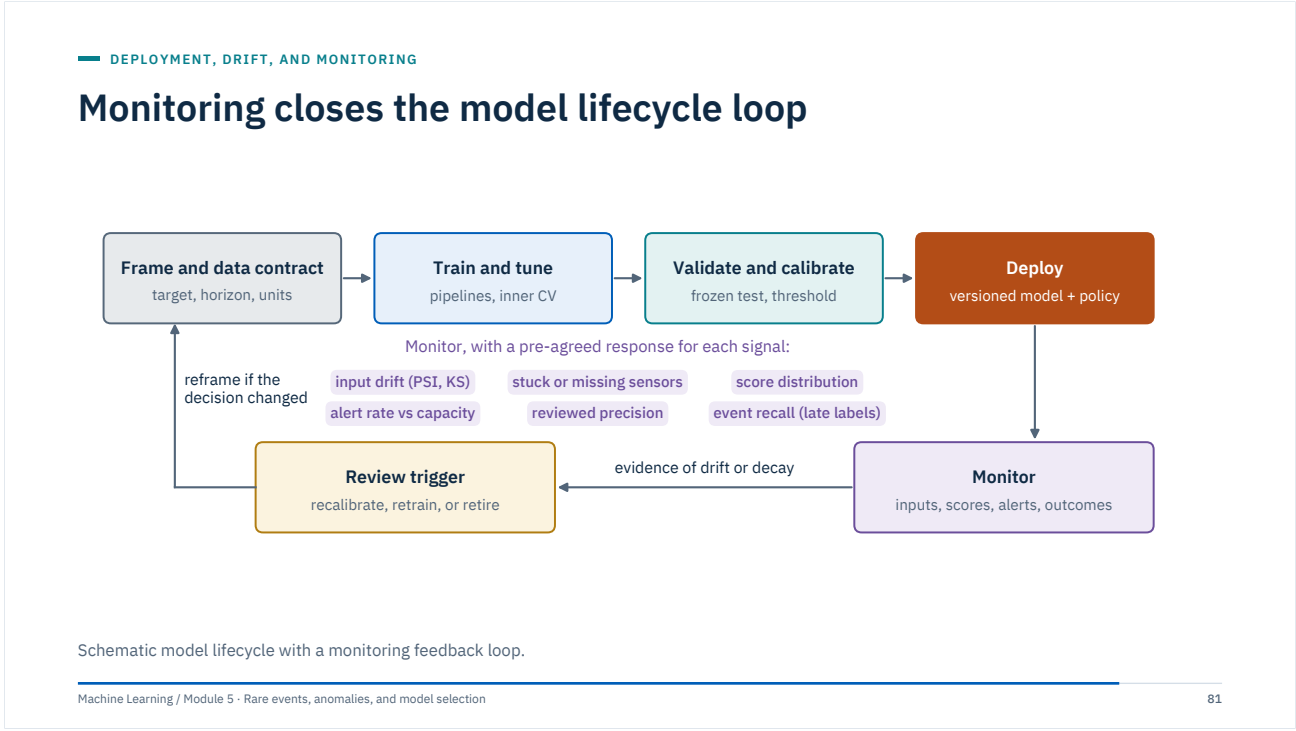
The population stability index (PSI) compares the share of rows in each bin of a reference distribution (here the deciles of the reference period) with the current share. Each bin contributes (a − e) × ln(a / e), which is never negative. Here the current period has 27% of its rows in the top reference decile instead of 10%, and PSI is 0.25.

The two-sample Kolmogorov–Smirnov statistic D is the largest vertical gap between the two cumulative distributions: 0.20 here, at about 126 bar.

Cautions. With thousands of rows, the KS p-value becomes tiny even for shifts too small to matter (here about 2.6 × 10^-40), so judge the size of D or PSI, not significance. A rule of thumb from credit scoring treats PSI below 0.1 as stable and above 0.25 as a large shift; it is only a convention, so calibrate alert levels on PSI between past stable periods. PSI depends on the binning, and empty bins need a small floor value.

Expect some inputs to move for harmless reasons (seasons, planned changes): drift is a reason to look, not a verdict. [3 min]

Monitoring closes the model lifecycle loop



Walk around the loop. Frame the decision and write the data contract: target, horizon, units, prediction time (Module 1). Train and tune with pipelines and inner cross-validation (Modules 1, 2 and today). Validate and calibrate on data untouched by tuning, and fix the threshold. Deploy a versioned model together with its alert policy and its model card (Module 2).

Monitor four families of signals, each with an owner and a pre-agreed response: inputs (drift, missing or stuck sensors), outputs (score distribution, alert rate against capacity), outcomes (precision of reviewed alerts, event recall once labels arrive) and operations (latency, failed runs).

A review trigger leads to one of three actions: recalibrate if only the probability scale moved, retrain if the relationship changed and new labels exist, or retire the model if the decision itself changed. If the question has changed, return to framing.

Ask: who in your organization would own each signal? Monitoring without an owner is only logging. [3 min]

A monitoring case is an experiment, not a promise

DEPLOYMENT, DRIFT, AND MONITORING

A monitoring case is an experiment, not a promise

Proposed use	Baseline	Evaluation question
ESP monitoring	Rules or logistic classifier	Event recall at a manageable alert rate
Pipeline investigation	Flow-balance residual	False alerts per unit exposure
Well-log quality review	Physical ranges and robust scores	Expert-reviewed ranking quality
Wind turbine performance	Binned power-curve residual	Share of alerts confirmed on inspection
Solar inverter monitoring	Comparison with neighbouring inverters	Confirmed losses per review hour

◆ Each row is a hypothesis to test on your own data; no field performance is claimed.

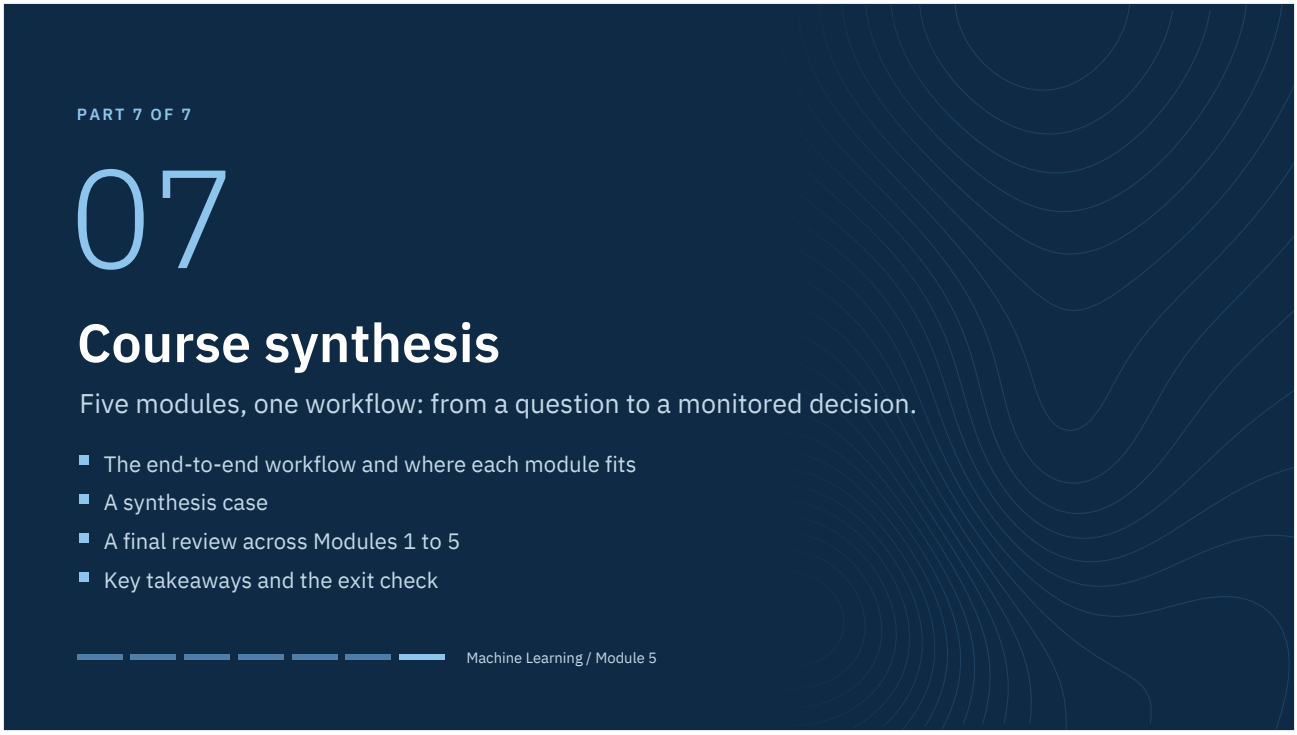
Machine Learning / Module 5 · Rare events, anomalies, and model selection 82

These are proposed application patterns. No field performance, detection lead time, or financial benefit is claimed by this slide. Additional engineering and operational validation is necessary before using a model for real decisions.

For each row, name the baseline the learned model must beat and the evaluation question that matters operationally. ESP monitoring: rules on current, intake pressure and temperature, or a logistic classifier; judged by event recall at an alert rate the team can handle. Pipeline investigation: a flow-balance residual; judged by false alerts per kilometre or per week. Well-log quality review: physical ranges and robust scores per zone; judged by how many top-ranked intervals an expert confirms. Wind: the residual from a power curve binned by wind speed on a reference period. Solar: each inverter compared with its neighbours under the same irradiance.

Ask each table to pick one row and name the first monitoring signal after deployment. [2 min]

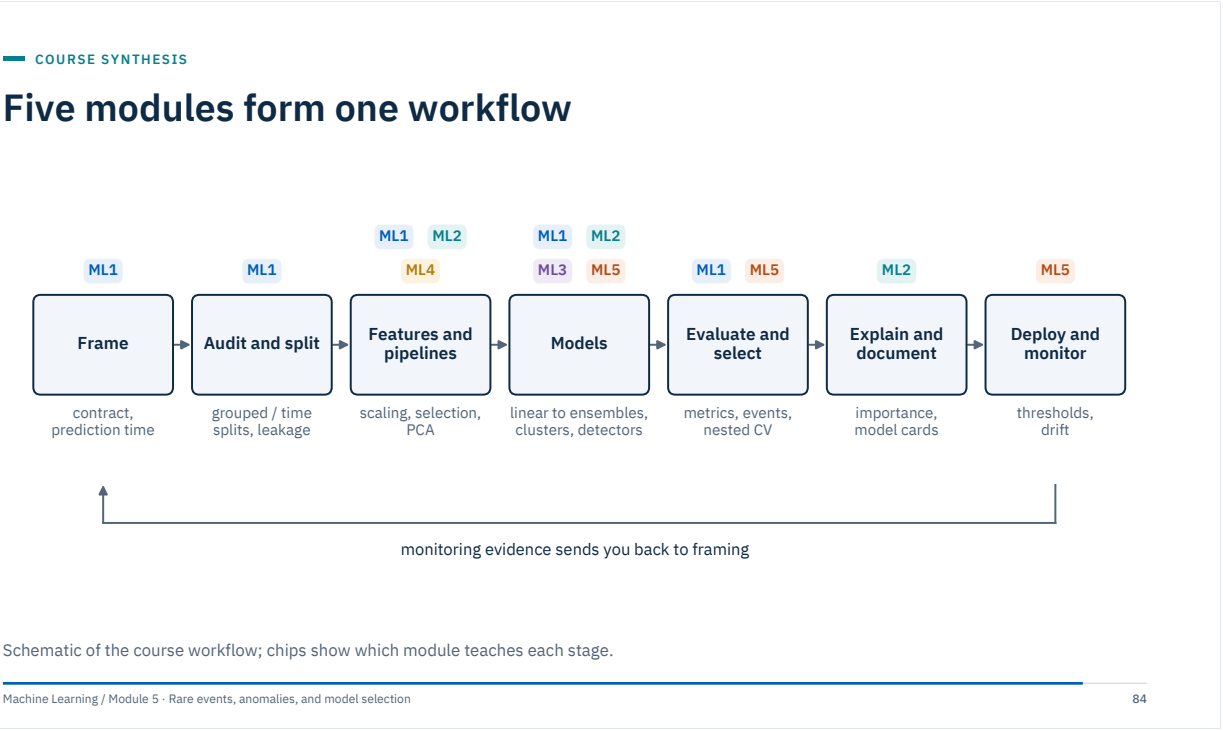
Course synthesis



The last 20 minutes step back from today's topics to the whole machine learning course. The aim is for learners to leave with one workflow they can apply to a new energy dataset, and to see where each module's tools belong in it.

Keep the pace brisk: two minutes for the map, four for the synthesis case, five for the final review, and the remaining time for takeaways and the exit check. Collect the exit-check answers; they are useful feedback on which ideas need reinforcement. [1 min]

Five modules form one workflow



Walk the map from left to right. Module 1, Supervised learning foundations, frames the problem (the prediction contract), audits the data, designs grouped and time-aware splits, and sets baselines and metrics. Module 2, Ensembles, interpretation, and pipelines, adds ensembles, pipelines with ColumnTransformer, interpretation (permutation importance, PDP, Shapley values) and model cards. Module 3, Clustering and mixture models, finds structure without labels (clusters and regimes), which today reappeared as the regimes that robust fences and detectors need. Module 4, Dimensionality reduction and feature selection, reduces dimension, and PCA reconstruction became one of today's detectors. Module 5, Rare events, anomalies, and model selection, adds anomaly detection, rare-event evaluation, calibration, tuning and monitoring.

The arrow back from monitoring to framing is the most important one: evidence after deployment changes the question, the data contract or the model.

The neural network course follows the same workflow with a new model family; nothing on this map is replaced, only the "models" box grows. [2 min]

Course synthesis: plan a new well-data project

PRACTICE

4 min Pairs

Course synthesis: plan a new well-data project

You receive a new well dataset and a request for a high-accuracy model.

1. State the first decision you must clarify.
2. Describe the baseline and the evaluation design.
3. Name the evidence needed before recommending deployment.

Require a concrete prediction contract, the independent sampling unit, leakage controls, a baseline, appropriate metrics, error analysis, and operational constraints. This is the final assessment of the machine learning course. Grade reasoning rather than library vocabulary.

Model answer. First decision: the prediction contract, meaning what is predicted (for example facies or a missing curve), for which unit (depth sample within a well), at what time, and what "high accuracy" must mean for the decision (a per-class recall, an error in physical units). Baseline and design: split by well with GroupKFold and keep a set of test wells untouched; baselines such as the majority class or a linear model; all preprocessing inside the pipeline; tuning in inner folds; metrics matched to the decision, per well and per class. Evidence before deployment: performance on unseen wells with intervals, error analysis by zone and by well, calibration if probabilities are used, a model card, and a monitoring plan with owners.

A strong answer mentions that "accuracy" may be the wrong target altogether. [4 min]

Final review: find the flaw in each plan

PRACTICE

5 min Groups

Final review: find the flaw in each plan

1. Rows from 40 wells are split at random; the test R^2 is 0.93.
2. A scaler and PCA are fitted on all rows before cross-validation.
3. k-means runs on raw GR (API) and RHOB (g/cc).
4. SMOTE is applied before the train/test split; recall is 0.95.
5. The alert threshold is tuned on the test set that reports precision.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 86

Groups of three or four take one item each, then report. One sentence per flaw and one per fix.

1. Random row splits put neighbouring depth samples from the same well in training and test, so the score measures interpolation within wells, not prediction for a new well. Fix: GroupKFold or a held-out set of wells (Module 1).

2. Fitting the scaler and PCA on all rows lets validation rows shape the preprocessing: leakage. Fix: put both steps in a Pipeline so they are refit inside each training fold (Modules 1 and 4).

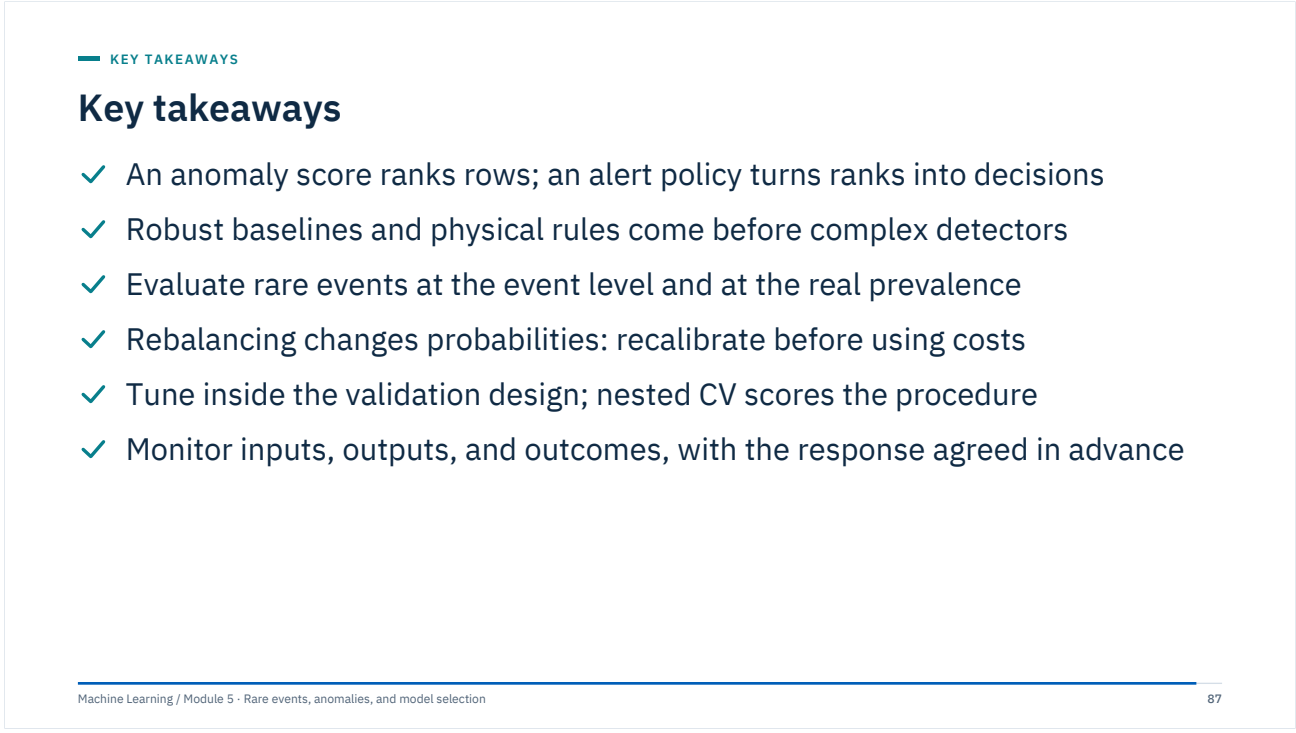
3. GR spans tens of API units while RHOB spans a fraction of a g/cc, so Euclidean distances, and therefore the clusters, are driven almost entirely by GR. Fix: standardize or use robust scaling first (Module 3).

4. SMOTE before the split creates synthetic training rows from test rows and changes the test prevalence, so recall is inflated. Fix: resample inside training folds only (Module 5, Part 4).

5. Tuning the threshold on the test set makes its precision an optimistic, selected number. Fix: choose the threshold on validation data or inner folds, freeze it, and test once (Module 5, Parts 3 and 5).

Common thread: every flaw lets information from the evaluation data into a choice. [5 min]

Key takeaways



Read each takeaway and link it to a moment in the day. Scores versus alerts: the contamination exercise and the Lab 2 policy. Baselines first: masking, fences and the physical-rule cards, and the Lab 1 finding that most IQR flags were shale. Event-level evaluation: the prevalence tree and the alert timeline. Calibration: the reliability diagram and the prior correction. Honest tuning: the best-of-50 optimism and nested cross-validation. Monitoring: the drift panels and the lifecycle loop.

If time allows, ask each learner which takeaway will change their next project; one or two answers make a good close. [2 min]

Exit check

PRACTICE

3 min · Individual

Exit check

1. Why can a detector with 80% recall and a 5% false-positive rate produce mostly false alerts?
2. What does nested cross-validation estimate that a tuned CV score does not?
3. Name one monitoring signal and the action it should trigger.

Machine Learning / Module 5 · Rare events, anomalies, and model selection 88

Collect written answers; they take about three minutes.

Answer 1: because the false-positive rate applies to the much larger ordinary population. With 1% prevalence, 80% recall and 5% FPR, 10,000 rows give 80 true and 495 false alerts, so precision is 13.9%.

Answer 2: nested cross-validation estimates the performance of the whole procedure (tuning and then fitting) on data that played no part in the tuning. A tuned CV score is the maximum over candidates of noisy estimates, so it is optimistic.

Answer 3: many good answers, provided the action is concrete. Examples: PSI above an agreed level on a key input triggers a data review and a coverage check; the alert rate exceeding the team's capacity for two weeks triggers a threshold review; the precision of reviewed alerts falling below a floor triggers recalibration or retraining; a sensor flatline triggers a maintenance ticket and removes that feature from scoring.

Thank the group; this closes the machine learning course. [3 min]

References and further study

FURTHER STUDY

References and further study

- Liu, Ting, Zhou (2008). Isolation Forest. IEEE International Conference on Data Mining.
- Breunig, Kriegel, Ng, Sander (2000). LOF: identifying density-based local outliers. ACM SIGMOD.
- Schölkopf et al. (2001). Estimating the support of a high-dimensional distribution. *Neural Computation* 13(7).
- Chandola, Banerjee, Kumar (2009). Anomaly detection: a survey. *ACM Computing Surveys* 41(3).
- Saito, Rehmsmeier (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE* 10(3).
- Bergstra, Bengio (2012). Random search for hyper-parameter optimization. *JMLR* 13.
- Cawley, Talbot (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *JMLR* 11.
- scikit-learn User Guide: outlier detection, calibration, tuning; imbalanced-learn: common pitfalls.

Use these references when adapting the lab or going deeper. Verify the distinction between score_samples, decision_function, and predict for each anomaly estimator in the official documentation before thresholding.

Further primary sources cited in today's notes: Iglewicz and Hoaglin (1993), How to Detect and Handle Outliers, ASQC; Breiman, Friedman, Olshen and Stone (1984), Classification and Regression Trees; Chawla et al. (2002), SMOTE, JAIR 16; Niculescu-Mizil and Caruana (2005), ICML; Dal Pozzolo et al. (2015), IEEE SSCI; Clopper and Pearson (1934), Biometrika 26; Baldi and Hornik (1989), Neural Networks 2; Jones, Schonlau and Welch (1998), Journal of Global Optimization 13; Snoek, Larochelle and Adams (2012), NeurIPS; Jamieson and Talwalkar (2016), AISTATS; Li et al. (2018), Hyperband, JMLR 18; Varma and Simon (2006), BMC Bioinformatics 7; Nadeau and Bengio (2003), Machine Learning 52; Gama et al. (2014), ACM Computing Surveys 46; Hastie, Tibshirani and Friedman (2009), The Elements of Statistical Learning, section 7.10. [1 min]

Reference: https://scikit-learn.org/stable/modules/outlier_detection.html ; https://scikit-learn.org/stable/modules/calibration.html ; https://scikit-learn.org/stable/modules/grid_search.html ; https://scikit-learn.org/stable/modules/model_evaluation.html ; https://imbalanced-learn.org/stable/common_pitfalls.html